

Machine learning : méthodes et solutions

Redha Moulla

Paris, 10 - 13 mars 2026

Objectifs de la formation

- Comprendre les différents modèles d'apprentissage.
- Modéliser un problème pratique sous forme abstraite.
- Identifier les méthodes d'apprentissage pertinentes pour résoudre un problème.
- Appliquer et évaluer les méthodes identifiées sur un problème.
- Faire le lien entre les différentes techniques d'apprentissage

Plan de la formation

- Aperçu de l'intelligence artificielle
- Apprentissage supervisé
- Apprentissage non supervisé
- Modèles bayésiens
- Deep learning
- Large Language Models (LLMs)

Qu'est-ce que l'intelligence artificielle ?

Définition littérale

1. Intelligence

Ensemble des fonctions mentales ayant pour objet la connaissance conceptuelle et relationnelle.

- Larousse

2. Artificielle

Qui est produit de l'activité humaine (opposé à la nature).

- Larousse

Qu'est-ce que l'intelligence ?

La notion d'intelligence recouvre plusieurs facultés cognitives :

- ① **Raisonnement** : La capacité à résoudre des problèmes et à faire des déductions logiques.
- ② **Apprentissage** : L'aptitude à acquérir de nouvelles connaissances et à s'améliorer grâce à l'expérience.
- ③ **Perception** : La compétence pour reconnaître et interpréter les stimuli sensoriels.
- ④ **Compréhension** : L'habileté à saisir le sens et l'importance de divers concepts et situations.
- ⑤ **Mémorisation** : La faculté de stocker et de rappeler des informations.
- ⑥ **Créativité** : Le pouvoir d'inventer ou de produire de nouvelles idées, de l'originalité dans la pensée.

Mais est-ce que l'intelligence est réductible à des facultés mesurables ?

Conférence de Dartmouth ?

Article

AI Magazine Volume 27 Number 4 (2006) (© AAAI)

A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence

August 31, 1955

John McCarthy, Marvin L. Minsky,
Nathaniel Rochester,
and Claude E. Shannon

■ The 1956 Dartmouth summer research project on artificial intelligence was initiated by this August 31, 1955 proposal, submitted by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon. The original typewritten document of 17 pages plus a title page. Copies of the typewritten are housed in the archives at Dartmouth College and Stanford University. The first 5 pages state the proposal, and the remaining pages give qualifications and interests of the four who proposed the study. In the interest of brevity, this article reproduces only the proposal itself, along with the short autobiographical statements of the proposers.

We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use lan-

guage, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer. The following are some aspects of the artificial intelligence problems.

1. Automatic Computers

If a machine can do a job, then an automatic calculator can be programmed to simulate the machine. The speed and memory capacities of present computers may be insufficient to simulate many of the higher functions of the human brain, but the major obstacle is not lack of machine capacity, but our inability to write programs taking full advantage of what we have.

2. How Can a Computer be Programmed to Use a Language

It may be speculated that a large part of human thought consists of manipulating words according to the rules of meaning and rules of inference. From this point of view, forming a generalization consists of admitting a new

"We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer."

L'intelligence artificielle selon John McCarthy

"It is the science and engineering of making intelligent machines, especially intelligent computer programs. It is related to the similar task of using computers to understand human intelligence, but AI does not have to confine itself to methods that are biologically observable."

— John McCarthy

"Il s'agit de la science et de l'ingénierie de la création de machines intelligentes, en particulier des programmes informatiques intelligents. Cela est lié à la tâche similaire d'utiliser les ordinateurs pour comprendre l'intelligence humaine, mais l'IA n'a pas besoin de se limiter aux méthodes qui sont biologiquement observables."

— John McCarthy

Le Test de Turing

Le Test de Turing, développé par Alan Turing en 1950, est une mesure de l'intelligence d'une machine. Le test évalue si une machine peut imiter l'intelligence humaine au point qu'un interrogateur humain ne peut pas distinguer de manière fiable si la communication vient d'une personne ou d'un programme informatique.

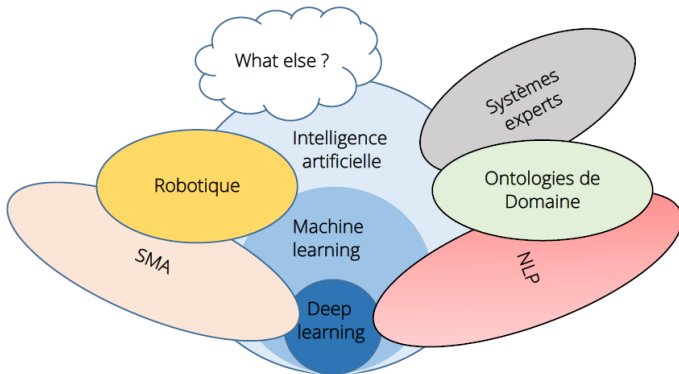
Les points clés du Test de Turing sont :

- Un interrogateur humain engage une conversation avec un humain et une machine, chacun étant caché de la vue de l'interrogateur.
- Si l'interrogateur ne peut pas déterminer systématiquement quelle est la machine, celle-ci est considérée comme ayant passé le test.
- Le test ne mesure pas la connaissance ou la capacité à être véridique, mais plutôt la capacité de reproduire le comportement humain.

“L'intelligence artificielle n'existe pas.”

— Luc Julia

Définition pragmatique de l'intelligence artificielle ?



IA forte vs IA faible

La distinction entre IA forte et IA faible se réfère à deux approches conceptuelles différentes dans le domaine de l'intelligence artificielle.

IA faible :

- Aussi connue sous le nom d'IA "étroite", elle est conçue pour effectuer des tâches spécifiques et ne possède pas de conscience.
- Les systèmes d'IA faible agissent et réagissent uniquement en fonction des instructions programmées et des algorithmes spécifiques.
- Exemples : assistants virtuels, systèmes de recommandation, reconnaissance vocale.

IA forte :

- Vise à créer des machines dotées de conscience, de compréhension et d'esprit, similaires à l'intelligence humaine.
- L'IA forte serait capable d'apprendre, de raisonner, de résoudre des problèmes et de prendre des décisions indépendamment.
- À ce jour, l'IA forte reste un objectif à atteindre, qui fait l'objet de recherches intensives.

IA connexionniste vs IA symbolique

Intelligence artificielle symbolique :

Systèmes basés sur des règles et des symboles pour imiter le raisonnement humain.

- Logique
- Ensemble de règles
- Orientée connaissance
- *Knowledge driven*

Intelligence artificielle connexionniste

: Modèles inspirés du cerveau humain pour apprendre des tâches à partir de données.

- Probabiliste
- Apprentissage machine
- Orientée données
- *Data driven*

Machine learning

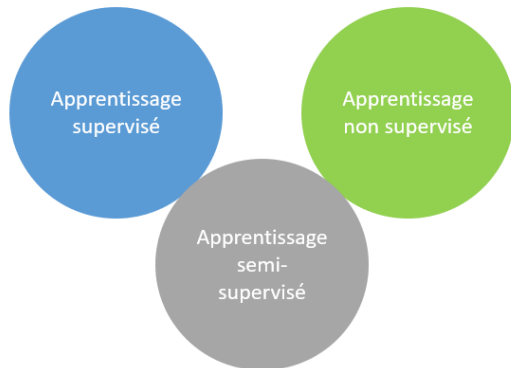
Définition de l'apprentissage automatique

L'apprentissage automatique est une branche de l'intelligence artificielle qui consiste à doter les machines de la capacité d'apprendre à partir de données sans que celles-ci ne soient explicitement programmées pour exécuter des tâches spécifiques.

Le machine learning englobe plusieurs types d'apprentissage :

- **Supervisé** : Les algorithmes apprennent à partir de données étiquetées pour faire des prédictions ou classifications.
- **Non supervisé** : L'apprentissage est effectué sur des données non étiquetées pour trouver des structures cachées.
- **Semi-supervisé** : Combine des éléments des deux premiers types en utilisant une petite quantité de données étiquetées et une grande quantité de données non étiquetées.
- **Par renforcement** : Les modèles apprennent à prendre des décisions en maximisant une récompense à travers des interactions.

Typologies d'apprentissage automatique



L'apprentissage supervisé

L'apprentissage supervisé consiste à apprendre un modèle qui associe une étiquette (*label*) à un ensemble de caractéristiques (*features*).

- **Inputs** : un jeu de données *annotées* pour entraîner le modèle.
 - Exemple : des textes (tweets, etc.) avec les *sentiment* associés, positifs ou négatifs.
- **Output** : une étiquette pour un point de donnée inconnu par le modèle.

L'apprentissage supervisé se décline lui-même en deux grandes familles :

- **La classification** : prédire une catégorie ou une classe.
 - Exemple : prédire l'étiquette d'une image (chat, chien, etc.), le sentiment associé à un texte, le centre d'intérêt d'un client à partir de ses commentaires, etc.
- **La régression** : prédire une valeur continue (un nombre réel typiquement).
 - Exemple : prédire le prix d'un appartement, la lifetime value d'un client, etc.

Classification

Exemple de classification : credit scoring

Âge	Revenu annuel (k€)	Historique de crédit	Nombre de cartes de crédit	Niveau d'éducation	Propriétaire immobilier (Oui/Non)	Label (y)
30	50	Bon	2	Licence	Oui	Accepté
45	80	Moyen	3	Master	Oui	Accepté
22	20	Mauvais	1	Bac	Non	Refusé
35	60	Bon	4	Licence	Oui	Accepté
40	70	Moyen	2	Bac+2	Non	Refusé

Exemple de régression : prédiction des prix des logements

Surface (m ²)	Nombre de chambres	Distance du centre-ville (km)	Année de construction	Quartier (Score 1-10)	Prix (k€) (y)
80	3	5	2010	8	300
120	4	10	2005	7	450
60	2	2	2020	9	200
150	5	15	1995	6	600
100	3	8	2015	7	400

L'apprentissage non supervisé

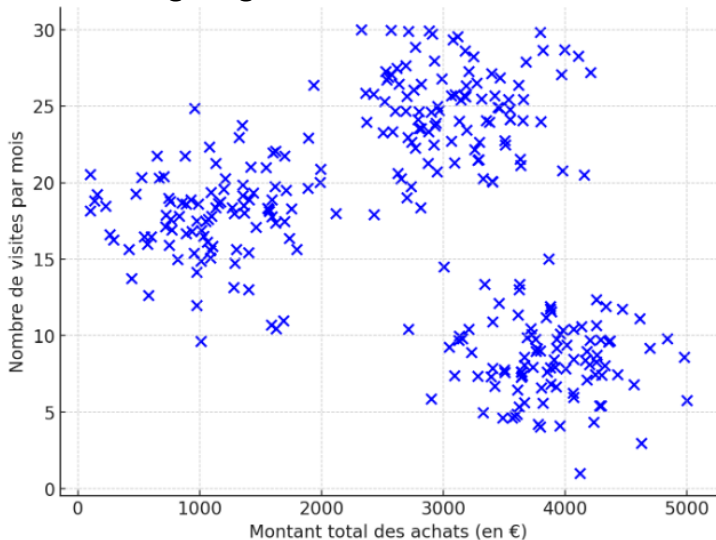
L'apprentissage non supervisé se réfère à l'utilisation de modèles d'apprentissage automatique pour identifier des patterns et des structures dans des données qui ne sont pas étiquetées.

Principales typologies de l'apprentissage non supervisé :

- **Clustering** : Regroupement de points de données similaires ensemble.
Exemple : segmentation de marché, regroupement social.
- **Détection d'anomalies** : Détecter des observations dont les caractéristiques sont inhabituelles par rapport à la majorité.

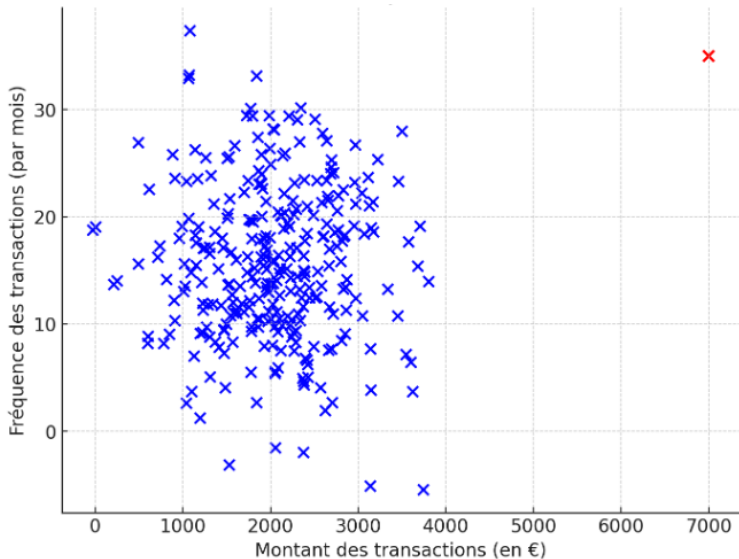
Clustering

Exemple de clustering : segmentation clients



Détection d'anomalies

Exemple de détection d'anomalies : fraude bancaire



L'apprentissage semi-supervisé

L'apprentissage semi-supervisé combine des éléments des approches supervisées et non supervisées. Il utilise un petit ensemble de données étiquetées et un plus grand ensemble de données non étiquetées pour former des modèles.

Cette méthode est particulièrement utile quand :

- Les données étiquetées nécessitent des ressources coûteuses pour les obtenir, mais les données non étiquetées sont abondantes.
- L'ajout d'un peu d'information étiquetée peut améliorer significativement la performance de modèles entraînés avec des données non étiquetées.

Les applications typiques incluent :

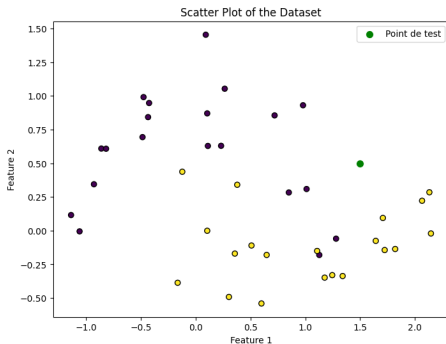
- Développement de systèmes de recommandation plus performants.
- Traitement de langage naturel et analyse de sentiment lorsque les annotations complètes ne sont pas disponibles.

L'apprentissage tente d'exploiter "le meilleur des deux mondes" de l'étiquetage et de la découverte de structure.

Apprentissage supervisé

L'apprentissage supervisé comme un problème d'induction

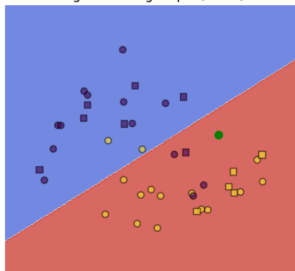
- **Définition** : L'apprentissage supervisé consiste à apprendre une fonction f qui mappe les entrées X aux sorties y , à partir d'un ensemble d'exemples d'entraînement (X, y) .
- **Induction** : Le modèle induit une règle générale à partir de données particulières, dans le but de généraliser à de nouvelles instances.
- **Problème de généralisation** : Comment garantir que le modèle apprend une règle qui s'applique à de nouvelles données ?



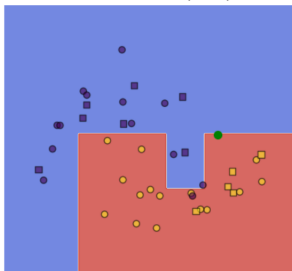
Une indétermination intrinsèque pour le choix du modèle

Il y a une infinité de manières d'induire un modèle à partir d'un échantillon de données d'entraînement.

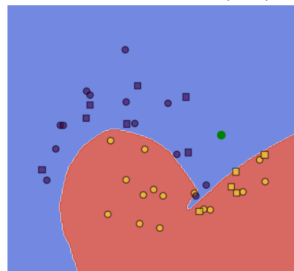
Régression Logistique (n=40)



Arbre de Décision (n=40)

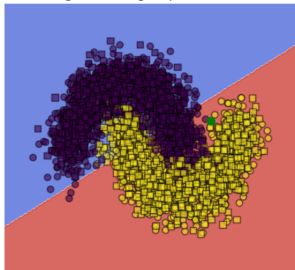


Réseau de Neurones Profond (n=40)

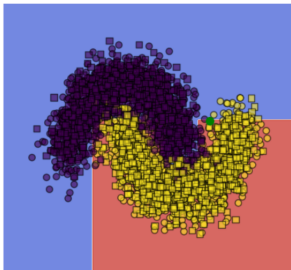


Sous-apprentissage et surapprentissage sur les données d'entraînement

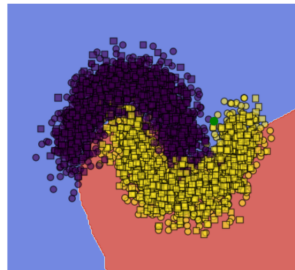
Régression Logistique (n=3000)



Arbre de Décision (n=3000)



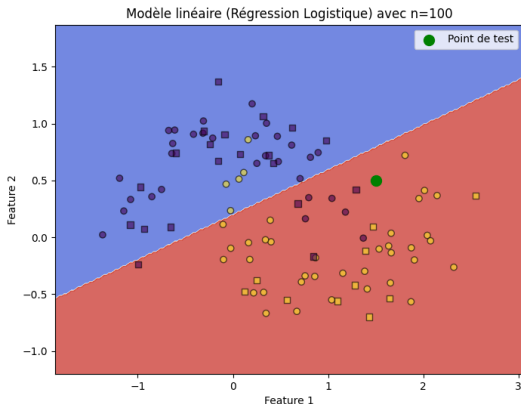
Réseau de Neurones Profond (n=3000)



Sous-apprentissage

Definition

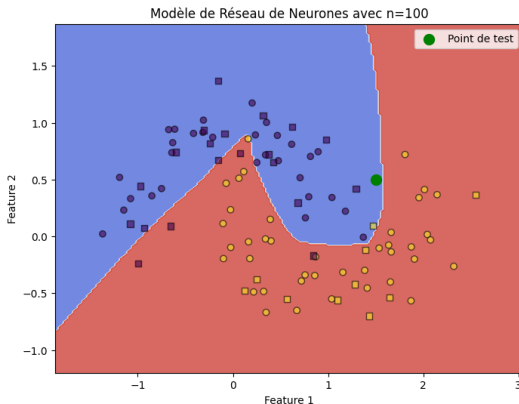
On dit qu'un modèle de machine learning est en régime de sous-apprentissage (underfitting) lorsqu'il n'arrive pas à capturer la complexité (l'information) présente dans le jeu de données d'entraînement.



Sur-apprentissage

Definition

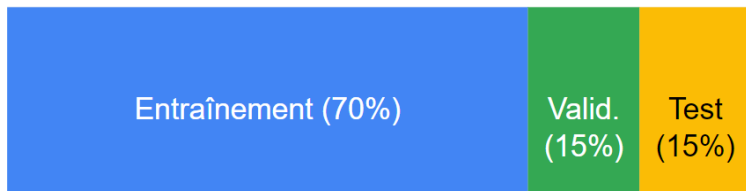
On dit qu'un modèle de machine learning est en régime de sur-apprentissage (overfitting) lorsqu'il n'arrive pas à généraliser à des données non encore observées, i.e. lorsqu'il est trop adapté aux données d'entraînement.



Sélection de modèle

Pour sélectionner le modèle le plus pertinent par rapport à une métrique donnée, on applique la méthodologie suivante :

- On partitionne le jeu de données disponible en trois parties : un jeu d'entraînement, un jeu de validation et un jeu de test.
- On entraîne M modèles sur le jeu d'entraînement.
- On évalue les performances respectives des M modèles sur le jeu de validation et on sélectionne le meilleur.
- Le modèle sélectionné est ensuite évalué sur le jeu de test. Idéalement, le jeu de test est ainsi utilisé une seule fois.



Exemple : sélection de modèle

Modèle	Précision Entraînement	Précision Validation
Régression Logistique	0.90	0.88
Arbre de Décision	0.95	0.92
Réseau de Neurones Profond	1	0.90

Table: Comparaison des précisions des modèles sur les jeux d'entraînement et de validation

Métriques de performance : régression

On dispose d'un certain nombre de métriques pour évaluer les performances des modèles de machine learning. Celles-ci peuvent être divisées en deux catégories.

Régression

- L'erreur quadratique moyenne (MSE) : elle est définie comme la moyenne des carrés des écarts entre les prédictions et les valeurs observées.

$$MSE = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2$$

- La racine carrée de l'erreur quadratique moyenne (RMSE) :

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2}$$

Métriques de performance : classification 1/2

Accuracy : L'accuracy est la métrique de base qui permet d'évaluer les performance d'un modèle de classification. Elle est définie comme :

$$\text{Accuracy} = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total de prédictions}}$$

Matrice de confusion : La matrice de confusion est une représentation permettant d'offrir plus de finesse par rapport à l'accuracy, notamment quand le jeu de données est déséquilibré (présence de classes majoritaires). Elle compare les prédictions du modèle avec les valeurs réelles et est structurée comme suit :

		Valeur Prédite	
		Positif	Négatif
Valeur Réelle	Positif	Vrai Positif (VP)	Faux Négatif (FN)
	Négatif	Faux Positif (FP)	Vrai Négatif (VN)

Métriques de performance : classification 2/2

A partir de la matrice de confusion, on peut dériver d'autres métriques :

- Précision : elle est définie comme la proportion des prédictions correctes parmi toutes les prédictions positives :

$$\text{Précision} = \frac{VP}{VP + FP}$$

- Rappel (recall) : il représente la proportion des vrais positifs correctement prédits par le modèle.

$$\text{Rappel} = \frac{VP}{VP + FN}$$

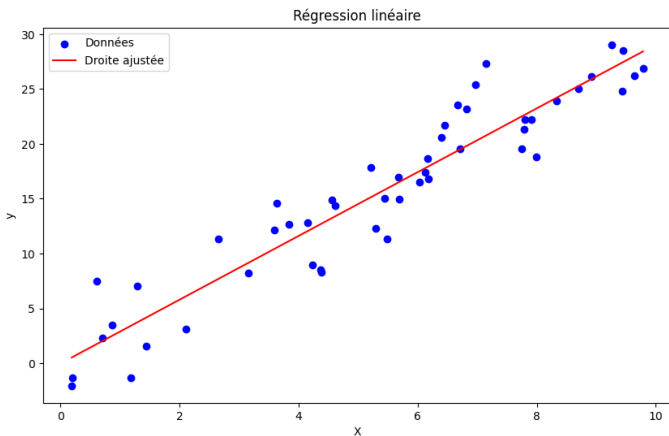
- Score F1 (F1-score) : Le score F1 est défini comme la moyenne harmonique de la précision et du rappel.

$$\text{Score F1} = 2 \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

Régression linéaire simple

Soit un ensemble de n observations $x_1, x_2, \dots, x_n$ avec les labels correspondants $y_1, y_2, \dots, y_n$, on cherche le modèle linéaire qui ajuste le mieux ces données.

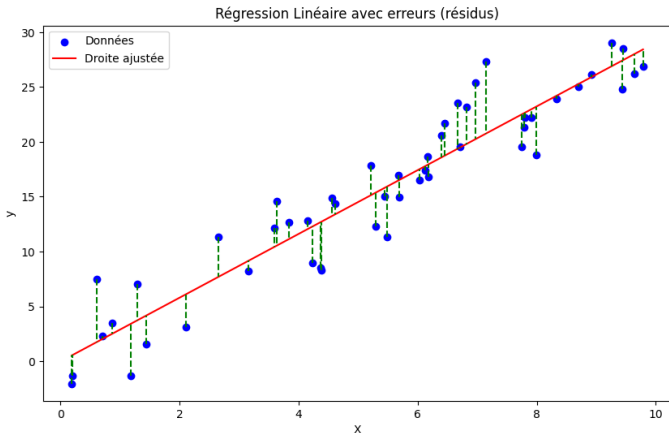
$$\hat{y} = \beta_0 + \beta_1 x$$



Résidus

Definition

Soit $\hat{y}_i = \beta_0 + \beta_1 x_i$ la i ième prédiction du modèle. Le i ième résidu, noté e_i , est alors défini comme l'erreur de prédiction sur i ième observation : $e_i = y_i - \hat{y}_i$.



Méthode des moindres carrés

On considère la somme des carrés des résidus, notée RSS :

$$RSS = e_1^2 + e_2^2 + \dots + e_n^2$$

L'approche par moindre carrés consiste à estimer les coefficients β_0 et β_1 en minimisant la RSS . Autrement dit :

$$\arg \min_{\beta_0, \beta_1} \left(\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 \right)$$

Le problème peut être résolu d'une manière analytique. On obtient :

$$\begin{aligned} \beta_1 &= \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \\ \beta_0 &= \bar{y} - \beta_1 \bar{x} \end{aligned} \tag{1}$$

Régression linéaire multiple

On considère n observations $X^1, X^2, \dots, X^n$ où chaque observation X^i est désormais un vecteur ayant p composantes (p variables explicatives).

$$X^i = \begin{pmatrix} x_1^i \\ x_2^i \\ \vdots \\ x_p^i \end{pmatrix}$$

La régression linéaire s'écrit alors :

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Les coefficients $\beta_0, \beta_1, \dots, \beta_p$ sont déterminés par la méthode des moindres carrés :

$$\arg \min_{\beta_0, \beta_1, \dots, \beta_p} \sum_{i=1}^n \left(y^i - \left(\beta_0 + \sum_{j=1}^p \beta_j x_j^i \right) \right)^2$$

L'équation normale

La régression linéaire peut être écrite sous une forme vectorielle :

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta}$$

où X est appelée matrice de design.

$$\mathbf{X} = \begin{pmatrix} 1 & x_1^1 & \dots & x_p^1 \\ \vdots & \vdots & \dots & \vdots \\ 1 & x_1^n & \dots & x_p^n \end{pmatrix}$$

La somme des carrés des résidus est donnée par :

$$RSS = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

On peut montrer que si la matrice de design X est de rang plein, alors :

$$\boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Régression polynomiale

La régression linéaire peut prendre en compte les dépendances non linéaires entre les variables explicatives $x_1, x_2, \dots, x_p$ et la variable expliquée y . Lorsque cette dépendance prend la forme d'un polynôme de degré d , la régression linéaire s'écrit alors :

$$\hat{y} = \beta_{00} + \sum_{j=1}^p \beta_{ij} x_j + \sum_{j=1}^p \beta_{ij} x_j^2 + \dots + \sum_{j=1}^p \beta_{ij} x_j^d$$

Les coefficients β_{ij} peuvent être de la même manière, avec la méthode des moindres carrés.

Remarque : On peut utiliser n'importe quelle fonction non linéaire pour transformer les variables explicatives ($\cos$, $\ln$, etc.). Le modèle reste tout de même linéaire (linéarité par rapport aux coefficients).

Choix de modèle

Il y a plusieurs manières d'évaluer la pertinence d'un modèle de régression linéaire. Le coefficient de détermination R^2 mesure l'ajustement du modèle. Il est donné par :

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

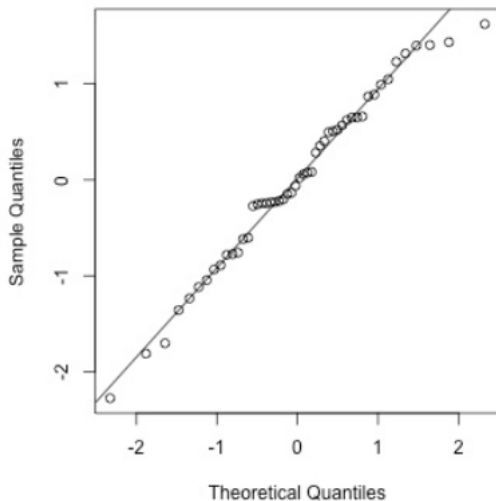
Pour une régression linéaire multiple, on préférera cependant le coefficient de détermination ajusté R_a^2 .

$$R_a^2 = 1 - \frac{n-1}{n-k-1} * (1 - R^2)$$

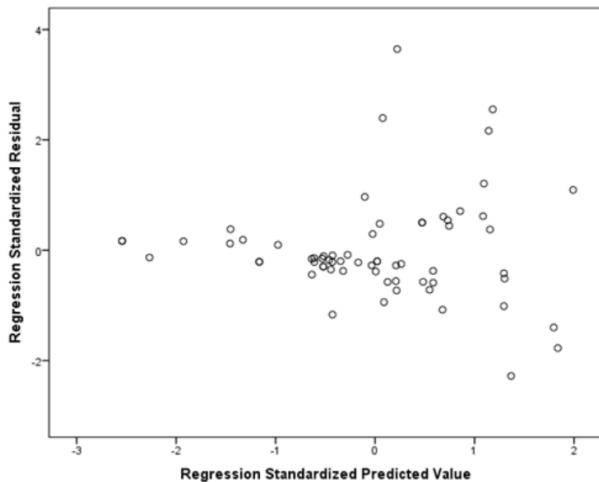
où n est le nombre d'observations et k le nombre de variables.

Des critères comme le AIC et le BIC sont également utilisés pour sélectionner un modèle.

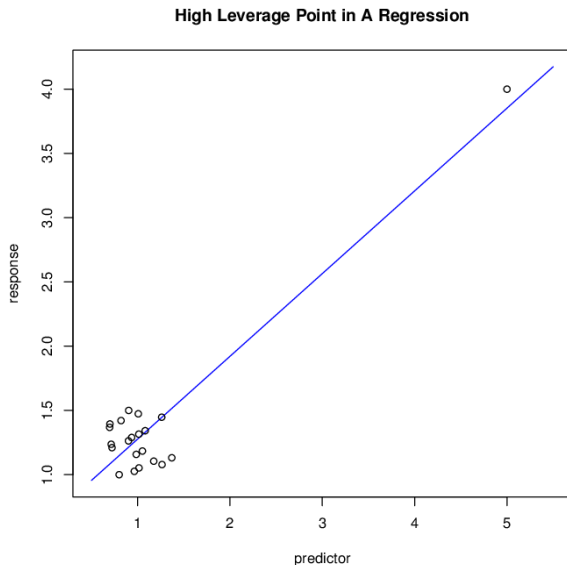
Diagnostic de la régression linéaire : normalité des résidus



Diagnostic de la régression linéaire : Homoscédasticité



Diagnostic de la régression linéaire : effet levier



Régression Ridge

La régularisation Ridge, dite également régularisation L_2 , permet de remédier au problème de surapprentissage en imposant une contrainte sur les coefficients β lors de la minimisation du risque empirique.

$$\arg \min_{\beta_0, \beta_1, \dots, \beta_p} \left(\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 \right)$$

sous la contrainte :

$$\sum_{j=1}^p \beta_j^2 \leq Cte$$

Le problème peut être écrit d'une manière plus compacte :

$$\arg \min_{\beta_0, \beta_1, \dots, \beta_p} \left(\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right)$$

où le paramètre λ contrôle la force la régularisation. Plus λ est grand, plus le modèle est régularisé.

Formulation vectorielle de la régression Ridge

La somme des carrés des résidus associée à la régression Ridge s'écrit de la manière suivante :

$$RSS = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda\boldsymbol{\beta}^T\boldsymbol{\beta}$$

Les coefficient $\vec{\beta}$ peuvent alors être calculés en minimisant la RSS .

$$\arg \min_{\beta_0, \beta_1, \dots, \beta_p} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda\boldsymbol{\beta}^T\boldsymbol{\beta}$$

On peut alors montrer que :

$$\boldsymbol{\beta} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

où $\mathbf{I}$ est la matrice identité de dimension $p \times p$.

Formulation vectorielle de la régression Ridge

La matrice de design X de dimensions $n \times n$ peut être décomposée en valeurs singulières (SVD) de la manière suivante :

$$\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^T$$

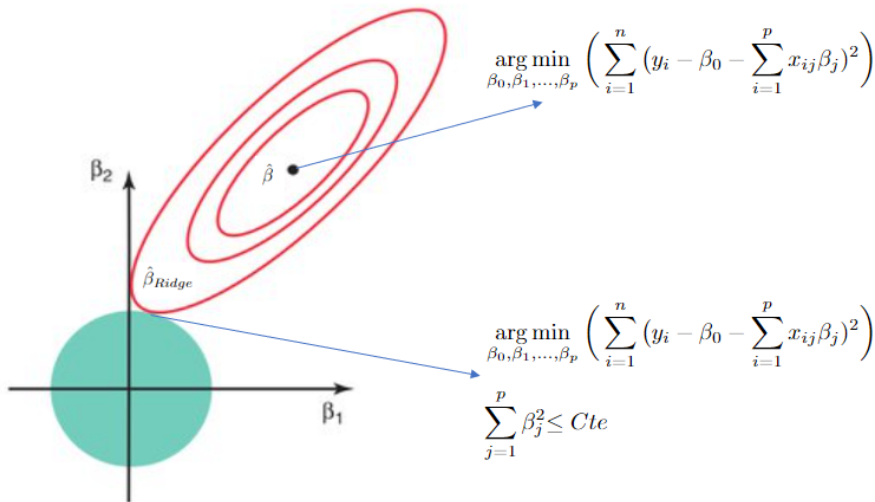
où $\mathbf{U}$ et $\mathbf{V}$ sont des matrices orthogonales de dimensions respectives $n \times p$ et $p \times p$.

On peut montrer que :

$$\begin{aligned}\mathbf{X}\beta &= \mathbf{X}(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y} \\ &= \sum_{j=1}^p u_j \frac{d_j^2}{d_j^2 + \lambda} u_j^T \vec{y}\end{aligned}\tag{2}$$

On peut noter que le paramètre de régularisation λ tend à réduire l'influence des variables explicatives associées à une faible valeur singulière.

Interprétation géométrique de régression Ridge



Régression Lasso

La régularisation Lasso, dite également régularisation L_1 , force les coefficients associés à des variables explicatives ayant une moindre importance vers zéro. C'est ainsi une technique de réduction de la dimensionalité du problème pour avoir un modèle avec peu de variable (plus simple et plus explicable).

La régression Lasso est formalisée de la manière suivante :

$$\arg \min_{\beta_0, \beta_1, \dots, \beta_p} \left(\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 \right)$$

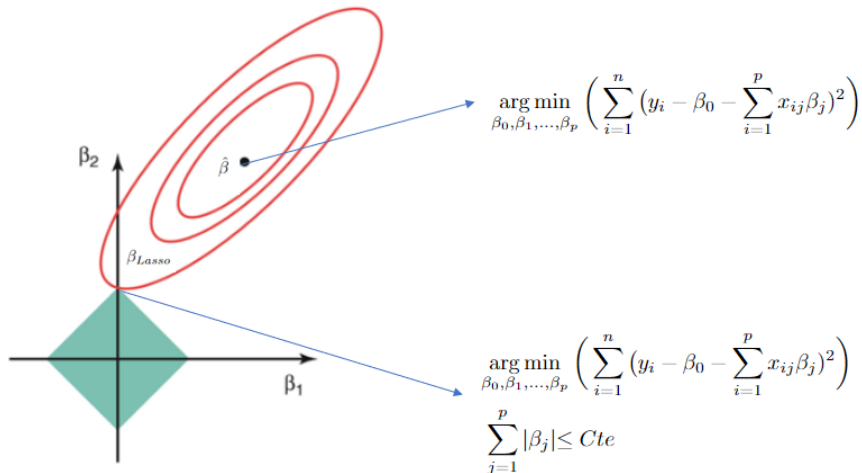
sous la contrainte :

$$\sum_{j=1}^p |\beta_j| \leq Cte$$

Où

$$\arg \min_{\beta_0, \beta_1, \dots, \beta_p} \left(\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j| \right)$$

Interprétation géométrique de régression Lasso



Elastic net

Elastic net combine les deux approches, Ridge et Lasso, pondérées avec un paramètre $\alpha \in [0, 1]$.

Le problème s'écrit ainsi :

$$\arg \min_{\beta_0, \beta_1, \dots, \beta_p} \left(\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 \right)$$

sous la contrainte :

$$\sum_{j=1}^p (1 - \alpha) |\beta_j| + \alpha \beta_j^2 \leq Cte$$

Ou

$$\arg \min_{\beta_0, \beta_1, \dots, \beta_p} \left(\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \left(\sum_{j=1}^p (1 - \alpha) |\beta_j| + \alpha \beta_j^2 \right) \right)$$

Régression logistique : introduction

La régression logistique est une technique d'analyse statistique utilisée pour modéliser la probabilité d'une variable dépendante binaire. C'est un cas particulier de modèle linéaire généralisé qui est utilisé pour des problèmes de classification.

Principes de la régression logistique :

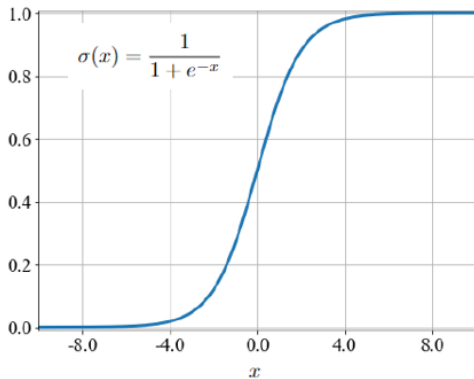
- **Variable dépendante** : On cherche la probabilité que la variable dépendante (y) appartienne à une classe (0 ou 1, vrai ou faux, succès ou échec). Autrement dit, on cherche à modéliser $P(y = 1)$ en fonction des variables dépendantes (explicatives) x .
- **Odds ratio** : Plus concrètement, on cherche à exprimer la côte anglaise (odd ratio) en fonction des variables dépendantes (x).

$$\ln \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Régression logistique : fonction sigmoïde

Après quelques simplifications, on peut écrire la probabilité $p(x)$ (la probabilité pour que y soit un succès par exemple) :

$$p(\mathbf{x}) = \frac{1}{1 + e^{-(\beta^T \mathbf{x})}}$$



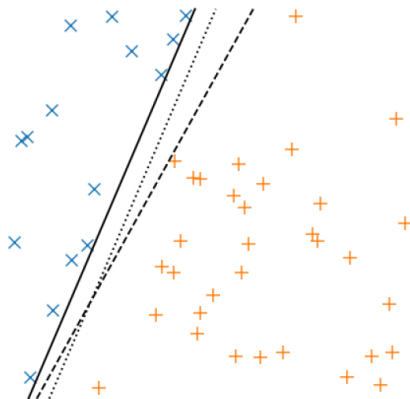
Calcul des coefficients de la régression logistique

Les coefficients de la régression logistique peuvent être calculés en minimisant le risque empirique par rapport à une fonction de coût sous forme d'entropie croisée :

$$\arg \min_{\beta_0, \beta_1, \dots, \beta_p} \left(- \sum_{i=1}^n y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i) \right)$$

Machines à vecteurs de support (SVM)

Considérons un problème de classification binaire. On recherche l'hyperplan séparateur qui maximise la marge γ entre les deux classes. La marge γ étant définie comme la distance entre cet hyperplan et les observations les plus proches.

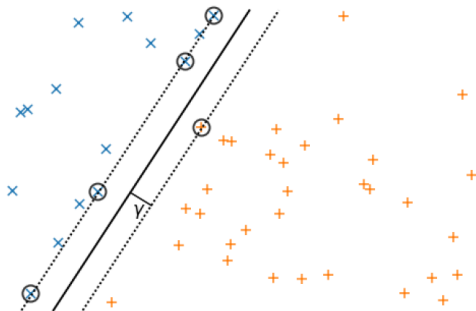


Position du problème

Soient n observations $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ et n poids associés $w_1, w_2, \dots, w_n$.

On cherche la fonction $(w^T x + b)$ telle que lorsque $h(x) \geq 1$ alors x appartient à la classe 1 et lorsque $h(x) \leq -1$ alors x appartient à la classe 0.

L'hyperplan d'équation $h(x) = 0$ est ainsi le plan séparateur pour les deux classes.



Formulation primale

La distance entre un point x_k et sa projection sur le plan séparateur est donnée par :

$$\gamma_k = y_k \frac{w^T x_k + b}{\|w\|}$$

Les poids w et le biais b peuvent être calculés en résolvant le problème d'optimisation suivant :

$$\arg \max_{w,b} \left(\frac{1}{\|w\|} \min_k (y_k (w_k^T x_k + b)) \right)$$

En pratique, il est plus simple de résoudre le problème équivalent :

$$\arg \min \frac{1}{2} \|w\|^2$$

sous la contrainte :

$$y_k (w^T x_k + b) \geq 1$$

Formulation duale 1/2

Le problème d'optimisation peut être résolu à l'aide des multiplicateurs de Lagrange α . Le lagrangien s'écrit :

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{k=1}^n \alpha_k \left(y_k (w^T x + b) \right)$$

En minimisant le lagrangien par rapport à w et b

$$\begin{aligned} \nabla_w L(w, b, \alpha) &= 0 \\ \frac{\partial L(w, b, \alpha)}{\partial b} &= 0 \end{aligned} \tag{3}$$

on obtient :

$$\begin{cases} \sum_{k=1}^n \alpha_k y_k x_k = w \\ \sum_{k=1}^n \alpha_k y_k = 0 \end{cases}$$

Formulation duale 2/2

En injectant les équations précédentes dans la formule du lagrangien, on obtient :

$$L(w, b, \alpha) = \sum_{k=1}^n \alpha_k - \frac{1}{2} \sum_{k,j} y_k y_j \alpha_k \alpha_j x_k^T x_j$$

Le problème duale s'écrit ainsi :

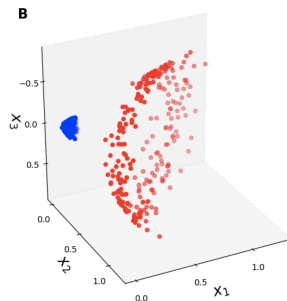
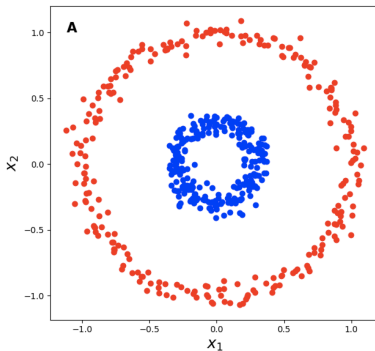
$$\arg \max_{\alpha} \left(\sum_{k=1}^n \alpha_k - \frac{1}{2} \sum_{k,j} y_k y_j \alpha_k \alpha_j x_k^T x_j \right)$$

sous les contraintes

$$\begin{cases} \alpha_k \geq 0, \quad k = 1, 2, \dots, n \\ \sum_{k=1}^n \alpha_k y_k = 0 \end{cases}$$

SVM à noyau

On considère un problème de classification non linéaire. L'astuce du noyau consiste à augmenter la dimension du problème, pour le résoudre ensuite avec un séparateur linéaire dans le nouvel espace.



Formulation duale des SVM à noyau

Le problème dual pour les SVM à noyau s'écrit alors :

$$\arg \max_{\alpha} \left(W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n y_i y_j \alpha_i \alpha_j K(x_i, x_j) \right)$$

sous les contraintes

$$\begin{cases} \alpha_k \geq 0, \quad k = 1, 2, \dots, n \\ \sum_{k=1}^n \alpha_k y_k = 0 \end{cases}$$

où la fonction noyau est donné par :

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

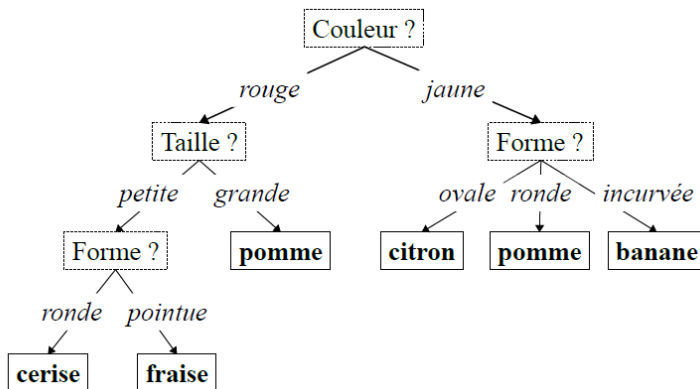
Exemples de noyaux

Il existe une variété de noyaux pour les SVM. Les plus utilisés étant les noyaux gaussien et polynomial.

- Noyau gaussien : $k(x, z) = \exp(-\frac{\|x-z\|^2}{2\sigma^2})$
- Noyau polynomial : $K(x, z) = (x^T z + 1)^d$

Arbre de décision

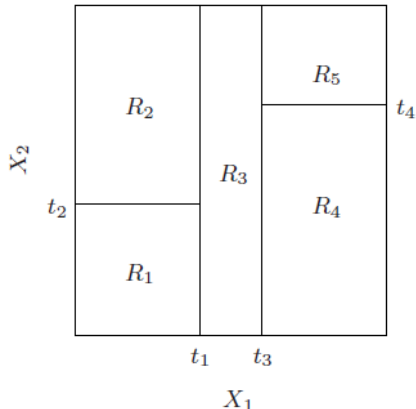
Les arbres de décisions sont des modèles dont le processus de décision est hiérarchique et prend la forme d'un arbre.



Entraînement des arbres de décision

Les arbres de décisions sont généralement entraînés à l'aide de la technique CART (Classification And Regression Trees).

Etant donné un ensemble d'observations $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, les arbres de décision partitionnent cet espace en plusieurs régions $R_1, R_2, \dots, R_m$.



Régression

Pour une régression, l'approche adoptée pour entraîner un arbre de décision consiste à chercher la meilleure variable x_j et le meilleur point de partitionnement s par rapport à cette variable en considérant le problème d'optimisation suivant :

$$\arg \min_{j,s} \left(\sum_{x_i \in R_l(j,s)} (y_i - y_l(j,s))^2 + \sum_{x_i \in R_r(j,s)} (y_i - y_r(j,s))^2 \right)$$

où $y_l(j,s)$ et $y_r(j,s)$ sont les moyennes des labels associées à chacune des deux régions formées par le partitionnement.

Il est important ici de noter que nous considérons comme critère d'optimisation la somme des erreurs quadratiques :

$$\sum_{i=1}^n (y_i - f(x_i))^2$$

Classification

L'approche pour la classification est similaire à celle pour la régression ; la fonction de coût quadratique est remplacée cette fois par une fonction mesurant l'impureté au niveau des feuilles (le degré d'hétérogénéité des labels).

On considère donc le problème d'optimisation suivant :

$$\arg \min_{j,s} \left(\sum_{x_i \in R_l(j,s)} \left(\frac{|R_l(j,s)|}{n} \text{Imp}(R_l(j,s)) + \frac{|R_r(j,s)|}{n} \text{Imp}(R_r(j,s)) \right) \right)$$

On définit la proportion d'exemples d'entraînement qui appartiennent à la classe c par :

$$p_{mk} = \frac{1}{|R_m|} \sum_{x_i \in R_m} \delta(y_i, k)$$

Différentes fonctions mesurant l'impureté peuvent être considérées :

- Erreur de classification : $1 - p_{mk}$
- Indice de Gini : $\sum_{k=1}^K p_{mk}(1 - p_{mk})$
- Entropie croisée : $-\sum_{k=1}^K p_{mk} \ln(p_{mk})$

Méthodes ensemblistes (bagging)

Le bagging vise à améliorer les performances des modèles en termes de robustesse (réduction de la variance). Il repose sur deux principes :

- Les prédictions issues d'un ensemble de modèles, même faibles, sont plus robustes que celles issues d'un seul modèle.
- L'échantillonnage aléatoire améliore la robustesse du modèle.

Soit un ensemble de n observations $\mathcal{D} = (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. Le bagging consiste à construire un modèle prédictif selon les étapes suivantes :

- On construit B échantillons en bootstrap (tirage avec remise) à partir de $\mathcal{D}$.
- On entraîne autant de modèles sur chaque échantillon.
- On combine les prédictions issues des B modèles (vote majoritaire pour la classification et calcul de la moyenne pour la régression).

Forêts aléatoires (random forests)

La technique des forêts aléatoires (random forests) consiste à appliquer une approche de type bagging sur les arbres de décision.

L'algorithme random forests suit cette procédure :

- Tirer par bootstrap B échantillons de tailles n à partir de l'ensemble D .
- Pour chaque échantillon tiré, construire un arbre en répétant les étapes suivantes jusqu'à atteindre n_{min} .
 - Tirer d'une manière aléatoire m variables parmi les p variables.
 - Sélectionner la meilleure variable avec le meilleur point de partitionnement.
 - partitionner le noeud en deux sous-branches.
- Agréger les arbres construits.

Les prédictions sont agrégées selon qu'il s'agisse de régression ou de classification :

- Régression : moyenne $f^B(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$
- Classification : vote majoritaire.

Remarques

- L'algorithme de random forests intègre nativement une forme de validation croisée. Les performances mesurées sur $\bigcup_{b_i \neq b_k}$ (out of bag ou OOB) sont souvent proches de celles que l'on pourrait mesurer avec une validation croisée.
- Le nombre de variable tirées pour chaque noeud est généralement donné par $\sqrt{p}$ pour la classification et $\frac{p}{3}$ pour la régression. Cet hyperparamètre dépend cependant du problème considéré.
- Lorsque le nombre de variable est élevé alors que le nombre de variables réellement pertinente est faible, la probabilité que les p variables sélectionnées pour chaque partitionnement incluent des variables pertinentes devient faible, et les performances du modèle en termes de généralisation peuvent se détériorer considérablement.
- L'algorithme random forests permet de restituer des informations sur l'importance des variable (feature importance).

Boosting

Le boosting est une méthode d'apprentissage ensembliste qui combine plusieurs modèles faibles pour créer un modèle global plus fort. Il se concentre sur la conversion d'apprenants faibles en apprenants forts.

Principes de Base :

- Construire séquentiellement des modèles faibles, généralement des arbres de décision.
- Chaque modèle suivant essaie de corriger les erreurs du modèle précédent.
- Les modèles sont pondérés en fonction de leur précision et combinés pour obtenir le modèle final.

Avantages :

- Amélioration de l'exactitude et de l'efficacité des prédictions.
- Bonne performance sur des ensembles de données variés.
- Réduction des risques de surapprentissage comparé aux modèles individuels.

Adaptive boosting (Adaboost)

Adaptive Boosting (Adaboost) : Méthode itérative pour améliorer un ensemble de modèles faibles.

Algorithme :

- ① Initialisez les poids des observations : $D_1(i) = \frac{1}{n}$.
- ② Pour $t = 1, 2, \dots, T$:
 - Entraînez un modèle faible $h_t(x)$.
 - Calculez l'erreur : $\epsilon_t = \sum D_t(i)[y_i \neq h_t(x_i)]$.
 - Calculez le poids du modèle : $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$.
 - Mettez à jour les poids : $D_{t+1}(i) = \frac{D_t(i)e^{-\alpha_t y_i h_t(x_i)}}{Z_t}$.
- ③ Modèle final : $H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$.

Remarques :

- Adaboost ajuste les poids des observations pour se concentrer sur les erreurs difficiles.
- Les modèles sont combinés en fonction de leur précision.

Gradient Boosting

Gradient Boosting : Construction itérative d'un modèle additif en minimisant une fonction de perte.

Processus Itératif :

- ① Commencez avec un modèle initial : $f_0(x)$.
- ② Pour $t = 1, 2, \dots, T$:
 - Calculez les résidus : $r_{ti} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{t-1}(x)}$.
 - Entraînez un modèle faible $h_t(x)$ sur r_{ti} .
 - Trouvez γ_t qui minimise $L(y_i, f_{t-1}(x_i) + \gamma h_t(x_i))$.
 - Mettez à jour : $f_t(x) = f_{t-1}(x) + \gamma_t h_t(x)$.
- ③ Modèle final : $f_T(x)$.

Remarques :

- Chaque modèle successif corrige les erreurs du modèle précédent.
- Les arbres de décision sont généralement utilisés comme modèles faibles.

Prétraitement des données

Prétraitement des données

Le prétraitement des données est une étape cruciale en machine learning qui vise à rendre les données plus appropriées pour le modèle. Cela inclut:

- Nettoyage des données
- Gestion des valeurs manquantes
- Normalisation et standardisation
- Encodage des variables catégorielles

Nettoyage des données

Le nettoyage des données implique de détecter et de corriger (ou supprimer) les erreurs et les incohérences pour améliorer la qualité des données. Cela inclut:

- Correction des erreurs de saisie
- Identification et traitement des valeurs aberrantes
- Suppression des doublons

Normalisation et standardisation

La mise à l'échelle des données est essentielle pour de nombreux algorithmes de machine learning.

- **Normalisation** redimensionne les valeurs dans un intervalle $[0, 1]$ ou $[-1, 1]$.
- **Standardisation** redimensionne les données pour qu'elles aient une moyenne de 0 et un écart type de 1.

Encodage des variables catégorielles

Les modèles de machine learning nécessitent des entrées numériques pour qu'elles soient traitées par la machine. Les variables catégorielles doivent donc être transformées en variables numériques. Il existe généralement deux approches :

- **Encodage One-Hot** crée une nouvelle colonne pour chaque catégorie.
- **Encodage ordinal** attribue un nombre unique à chaque catégorie selon un ordre spécifique.

Gestion des valeurs manquantes

Les valeurs manquantes représentent l'absence d'information dans un ensemble de données et constituent un défi courant en machine learning et analyse de données. Elles peuvent survenir pour diverses raisons, telles que :

- Erreurs de saisie des données
- Perte de données lors de la transmission
- Non-réponse dans les enquêtes ou questionnaires
- Suppression intentionnelle de données pour des raisons de confidentialité

La gestion appropriée des valeurs manquantes est cruciale pour maintenir la qualité et la fiabilité des modèles prédictifs. Elle implique des techniques telles que l'imputation, la suppression des observations manquantes, ou l'utilisation de modèles capables de gérer directement les données manquantes.

Typologies des données manquantes

Comprendre la nature des données manquantes est crucial pour choisir la méthode de traitement appropriée.

- **MCAR (Missing Completely At Random)** : La probabilité qu'une donnée soit manquante est la même pour toutes les observations. L'absence de données est totalement indépendante des données observées ou manquantes.
- **MAR (Missing At Random)** : La probabilité qu'une donnée soit manquante dépend des données observées et non des données manquantes elles-mêmes.
- **MNAR (Missing Not At Random)** : La probabilité qu'une donnée soit manquante dépend des données manquantes elles-mêmes.

La distinction entre ces catégories influence la stratégie d'imputation et l'analyse des données.

Stratégies de base pour les données manquantes

Selon la typologie, différentes stratégies peuvent être adoptées :

- **Pour MCAR et MAR :**

- Suppression de lignes ou de colonnes
- Imputation simple (moyenne, médiane, mode, etc.)

- **Pour MNAR :**

- Techniques d'imputation plus sophistiquées
- Modélisation spécifique pour estimer les valeurs manquantes

L'identification du type de données manquantes aide à choisir la méthode la plus adaptée et à minimiser le biais introduit par l'imputation.

Imputation simple vs. Imputation multiple

- **Imputation simple :**

- Remplace les valeurs manquantes par une estimation (moyenne, médiane, etc.).
- Facile à implémenter mais ne tient pas compte de l'incertitude autour des valeurs imputées.

- **Imputation multiple :**

- Génère plusieurs ensembles complets de données en remplaçant les valeurs manquantes par un ensemble de valeurs plausibles.
- Permet d'estimer l'incertitude autour des imputations et offre des estimations plus robustes.

L'imputation multiple est particulièrement utile pour les données MAR et MNAR, où l'incertitude autour des valeurs manquantes doit être prise en compte.

Techniques avancées d'imputation

Des techniques avancées peuvent mieux gérer la complexité des données manquantes :

- **Imputation KNN (k-Nearest Neighbors)** : Utilise les k observations les plus similaires pour imputer les valeurs manquantes.
- **Imputation par modèles prédictifs** : Utilise des modèles comme les arbres de décision, forêts aléatoires ou réseaux de neurones pour prédire les valeurs manquantes.
- **Imputation par chaînes de Markov Monte Carlo (MCMC)** : Une approche probabiliste qui génère des valeurs plausibles pour les données manquantes en se basant sur leur distribution.

Ces méthodes tentent de modéliser la structure sous-jacente des données pour une imputation précise et pour minimiser le biais.

Considérations finales sur le traitement des données manquantes

- Identifier la typologie des données manquantes est essentiel pour choisir la méthode d'imputation appropriée.
- Les méthodes d'imputation doivent être choisies en tenant compte de l'impact sur la distribution des données et sur les analyses ultérieures.
- L'imputation multiple est recommandée pour obtenir des estimations plus robustes et évaluer l'incertitude autour des valeurs imputées.
- L'exploration des données et la compréhension du contexte sont cruciales pour interpréter correctement les raisons derrière les données manquantes et pour choisir la méthode d'imputation la plus adéquate.
- Il est important de documenter le processus d'imputation et d'évaluer l'impact de différentes stratégies sur les résultats du modèle.

Considérer le traitement des données manquantes comme une composante intégrale de la préparation des données peut améliorer significativement la qualité et la fiabilité des analyses de machine learning.

Apprentissage non supervisé

Apprentissage non supervisé

Dans l'apprentissage non supervisé, on considère n observations sans labels. On s'intéresse fondamentalement à la probabilité jointe de ces observations.

On peut distinguer deux grandes catégories d'apprentissage non supervisé :

- Clustering (partitionnement) : cela consiste à partitionner les n observations en K groupes pertinents (généralement le critère de pertinence a une signification d'un point de vue métier).
- Réduction de dimension : il s'agit de trouver une représentation des données originelles dans un nouvelles espace de plus petite dimension. Cela peut être effectué à différentes fins : visualisation des données, compression des données, amélioration des performances du modèles (modèles plus robuste, plus explicable, etc.).

Méthode des k-moyennes (k-means)

Soit un ensemble de n observations $(x_1, x_2, \dots, x_n)$. La méthode des k-means vise à partitionner cet ensemble selon k groupes en minimisant la variance globale à l'intérieur des clusters. Plus formellement, le problème des k-means s'écrit :

$$\arg \min_{\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_K} \sum_{k=1}^K \sum_{x \in \mathcal{C}_k} \|x - u_k\|^2$$

Une telle optimisation est cependant difficile et très coûteuse en temps.

Algorithme de Lloyd

Considérons n observations $x_1, x_2, \dots, x_n$ et un nombre déterminé K de clusters, l'algorithme de Lloyd consiste à exécuter les étapes suivantes :

- On choisit aléatoirement k centroïdes parmi les n observations.
- On affecte chaque observation x_i au cluster dont le centroïde est plus proche.

$$k(x_i) = \arg \min_{k=1,2,\dots,K} \|x_i - u_k\|$$

- recalculer les centroïdes de chaque cluster avec la nouvelle configuration.

$$\mu_k = \frac{1}{C_k} \sum_{x_i \in C_k} x_i$$

- On répète la procédure jusqu'à convergence (jusqu'à ce que les centroïdes soient stables).

L'algorithme de Lloyd tente de regrouper les données en clusters en minimisant les distances entre les points d'un même cluster tout en maximisant les distances entre points appartenant à différents clusters.

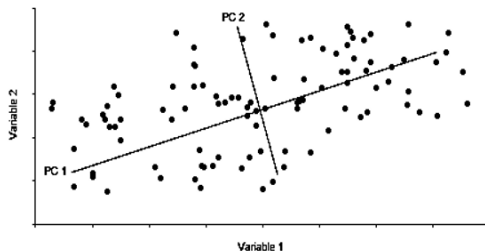


Remarques

- L'algorithme des k-means étant basé sur une distance euclidienne, il est nécessaire de normaliser les données avant de l'exécuter.
- L'algorithme des k-means est très sensible aux données aberrantes (outliers). Il faut donc considérer les données d'une manière attentive. Cependant, cela permet également d'utiliser l'algorithme des k-means pour la détection automatique des outliers.
- Les centroïdes étant initialisés d'une manière aléatoire, les clusters obtenus ne sont pas stables ; les clusters peuvent changer d'une exécution à l'autre. Il existe cependant une variante plus stable, appelée k-means++, qui permet de sélectionner les centroïdes d'une manière semi-aléatoire.
- Il est possible de partitionner les données avec une métrique plus générale que la distance euclidienne. On peut définir un algorithme k-means à noyau sur un espace de Hilbert pour aller au-delà de la métrique euclidienne.
- K-means n'est pas adapté aux données en grande dimension.

Analyse en Composantes Principales (ACP)

L'Analyse en Composantes Principales (ACP) est une technique statistique de réduction de dimensionnalité. Elle transforme les données en un nouveau système de coordonnées où la plus grande variance est capturée sur les premiers axes, appelés composantes principales.



Formulation mathématique de l'ACP

Soit X une matrice de données de dimension $n \times p$ (n observations, p variables), centrée (moyenne nulle). L'ACP cherche à trouver les vecteurs propres et les valeurs propres de la matrice de covariance $C = \frac{1}{n-1} X^T X$.

Les composantes principales sont données par les vecteurs propres u_k de C , ordonnés par leurs valeurs propres correspondantes λ_k en ordre décroissant.

La k -ième composante principale de l'ensemble de données est donnée par :

$$Z_k = Xu_k$$

Les valeurs propres λ_k représentent la variance expliquée par chaque composante principale.

Décomposition en valeurs propres

La matrice de covariance C peut être décomposée comme suit :

$$C = VLV^T$$

où $V = [u_1, u_2, \dots, u_p]$ est la matrice des vecteurs propres et L est une matrice diagonale des valeurs propres λ_k .

La contribution de chaque composante principale à la variance totale est donnée par :

$$\frac{\lambda_k}{\sum_{i=1}^p \lambda_i}$$

Interprétation des composantes principales

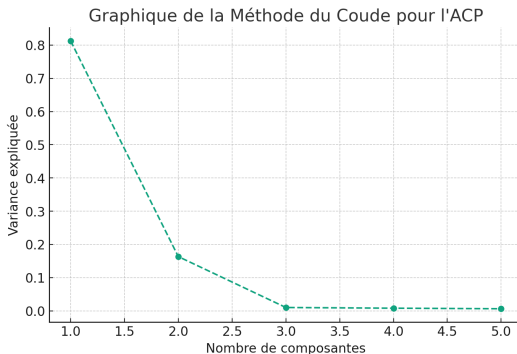
Chaque composante principale est une combinaison linéaire des variables d'origine. L'interprétation des composantes principales peut être réalisée en examinant les coefficients (charges) de ces combinaisons linéaires.

- Plus le coefficient absolu d'une variable est grand dans une composante, plus cette variable contribue à la variance capturée par cette composante.
- La direction et la magnitude des composantes principales peuvent être visualisées sur un biplot.

Choix du nombre de composantes principales

Le nombre de composantes à retenir est déterminé en fonction du pourcentage de variance totale que l'on souhaite expliquer.

- La méthode du coude (Scree plot) : un graphique montrant la proportion de la variance expliquée en fonction du nombre de composantes.
- Critère de Kaiser : retenir les composantes avec une valeur propre supérieure à 1.



Réduction de dimensionnalité

L'ACP est souvent utilisée pour réduire la dimensionnalité des données en conservant les composantes qui capturent la majorité de la variance.

- Cela permet une visualisation simplifiée des données en 2D ou 3D.
- La réduction de dimensionnalité peut également aider à améliorer l'efficacité des algorithmes d'apprentissage supervisé.

L'ACP a de nombreuses applications dans différents domaines :

- Analyse de données en biologie, finance, marketing.
- Traitement d'images et de signaux.
- Reconnaissance de motifs et classification.

Limitations de l'ACP

Bien que l'ACP soit un outil puissant, elle présente certaines limitations :

- Sensibilité aux outliers.
- Difficulté d'interprétation des composantes si les variables sont fortement corrélées.
- La réduction de dimensionnalité peut entraîner une perte d'informations importantes.

Isolation Forest : Principe et Application

L'Isolation Forest est une technique de détection d'anomalies basée sur l'isolement des observations. Son efficacité repose sur l'hypothèse que les anomalies sont "faciles à isoler" par rapport aux observations normales.

- Fonctionne en construisant des arbres d'isolement à partir de sous-ensembles de données.
- Isoler une observation signifie la séparer des autres par des divisions aléatoires de l'espace des caractéristiques.
- Moins de divisions sont nécessaires pour isoler une anomalie, ce qui constitue le fondement du score d'anomalie.

Construction des arbres d'isolement

Les arbres d'isolement sont construits de manière récursive :

- ① Sélection aléatoire d'un sous-ensemble de données.
- ② Choix aléatoire d'une caractéristique et d'une valeur de seuil pour diviser le sous-ensemble.
- ③ Répétition des divisions jusqu'à l'isolement des observations ou atteinte d'une limite de profondeur prédéfinie.

Chaque arbre est ainsi unique, offrant une perspective différente sur les données.

Calcul du score d'anomalie

Le score d'anomalie reflète la facilité avec laquelle une observation est isolée :

- Basé sur la longueur du chemin moyen (nombre de divisions) pour isoler une observation dans les arbres d'isolement.
- Un chemin court indique une forte probabilité d'anomalie.
- Le score est normalisé pour se situer entre 0 et 1, où les valeurs proches de 1 indiquent des anomalies.

Formule du score d'anomalie :

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

Où $s(x, n)$ est le score d'anomalie de l'observation x dans un échantillon de n observations, $E(h(x))$ est la longueur moyenne du chemin dans l'ensemble des arbres d'isolement, et $c(n)$ est le facteur de normalisation.

Avantages et Limitations de l'Isolation Forest

Avantages :

- Efficace sur de grands ensembles de données avec une complexité temporelle linéaire.
- Peu sensible aux paramètres, facilitant sa configuration et son utilisation.
- Performant pour détecter les anomalies réelles, même dans des ensembles de données bruités.

Limitations :

- La performance peut varier en fonction du choix de la taille de l'échantillon et du nombre d'arbres.
- Peut être moins efficace pour identifier des groupes d'anomalies denses.

Applications de l'Isolation Forest

L'Isolation Forest est largement utilisé dans des domaines variés pour sa capacité à identifier rapidement les anomalies :

- Détection de fraude financière.
- Maintenance prédictive dans l'industrie.
- Sécurité informatique et détection d'intrusions.
- Analyse de données médicales pour identifier les cas atypiques.

Sa flexibilité et son efficacité en font un outil précieux pour de nombreux scénarios d'analyse de données.

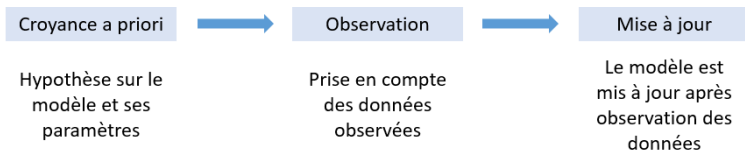
Méthodes bayésiennes

Introduction aux méthodes bayésiennes

Les méthodes bayésiennes fournissent un cadre complet pour construire des modèles prédictifs et gérer les incertitudes liées à leurs paramètres. Elles sont basées sur le théorème de Bayes pour mettre à jour les connaissances ou croyances antérieures avec de nouvelles données observées.

L'approche bayésienne repose sur deux concepts :

- 1 **La probabilité a priori** : La connaissance ou l'hypothèse initiale sur les paramètres avant d'observer les données. Elle reflète les croyances préalables sur les paramètres du modèle.
- 2 **La probabilité a posteriori** : Les paramètres du modèle (plutôt leur probabilité) sont mis à jour des paramètres après avoir pris en compte les nouvelles données observées en utilisant le théorème de Bayes.



Formulation mathématique de l'approche bayésienne

Concrètement, la probabilité du modèle est mise à jour en utilisant le théorème de Bayes :

$$P(\theta|\text{données}) = \frac{P(\text{données}|\theta) \cdot P(\theta)}{P(\text{données})}$$

où :

- $P(\theta|\text{données})$ est la probabilité a posteriori des paramètres θ étant donné les données.
- $P(\text{données}|\theta)$ est la vraisemblance des données observées sous l'hypothèse des paramètres θ .
- $P(\theta)$ est la probabilité a priori des paramètres avant d'observer les données.
- $P(\text{données})$ est la probabilité marginale des données, agissant comme une constante de normalisation.

Le calcul de $P(\text{données})$, la constante de normalisation, est généralement très délicat et nécessite des approximations.

Estimation des paramètres dans les modèles bayésiens

L'entraînement d'un modèle bayésien revient à estimer les distributions a posteriori des paramètres sur la base des données observées. Cette distribution a posteriori capture notre compréhension et notre incertitude concernant les paramètres après avoir pris en compte les informations fournies par les données.

$$\theta_{\text{post}} = \arg \max_{\theta} P(\theta | \text{données})$$

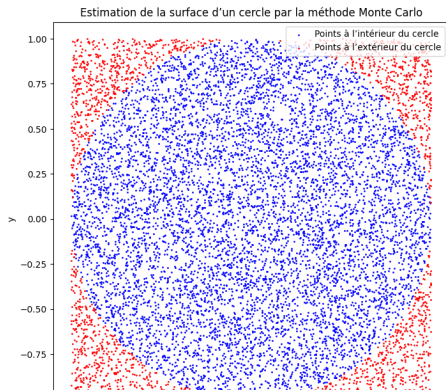
Les méthodes d'estimation des paramètres incluent :

- ➊ **Méthodes analytiques** : Dans certains cas, la distribution a posteriori peut être calculée directement à partir de formules analytiques, souvent lorsque le modèle prior et la vraisemblance sont conjugués.
- ➋ **Méthodes d'échantillonnage** : Lorsque les solutions analytiques ne sont pas réalisables, des techniques d'échantillonnage telles que la chaîne de Markov Monte Carlo (MCMC) sont utilisées pour générer des échantillons de la distribution a posteriori.
- ➌ **Approximation variationnelle** : Une alternative aux méthodes d'échantillonnage, où l'objectif est de trouver une distribution simple qui se rapproche au mieux de la distribution a posteriori complexe.

Intuition derrière les techniques de Monte Carlo

Le graphique ci-dessus illustre l'utilisation de la méthode de Monte Carlo pour estimer la surface d'un cercle unitaire. En générant aléatoirement des points dans le carré $[1, 1] \times [1, 1]$ et en comptant la proportion de ces points qui se trouvent à l'intérieur du cercle (points bleus), on peut estimer la surface du cercle.

L'estimation de la surface du cercle obtenue à partir de cette simulation est de 3.1492, ce qui est proche de la valeur théorique de $\pi = 3.1416$.



Régression bayésienne

La régression bayésienne est une extension des méthodes de régression classiques qui incorpore l'incertitude dans les estimations des paramètres de régression. En utilisant l'approche bayésienne, chaque paramètre de régression est traité comme une variable aléatoire avec sa propre distribution a priori, qui est mise à jour en une distribution a posteriori en tenant compte des données observées.

La formulation de base de la régression bayésienne linéaire est :

$$y = \mathbf{X}\beta + \epsilon$$

où y est le vecteur de réponse, $\mathbf{X}$ est la matrice de conception, β est le vecteur des coefficients de régression, et ϵ est le terme d'erreur, souvent supposé suivre une distribution normale avec une moyenne de zéro.

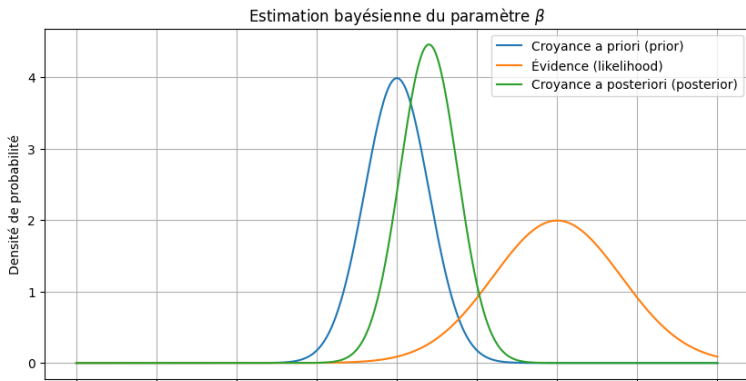
Le processus d'estimation bayésienne se concentre sur la détermination de la distribution a posteriori des coefficients β :

$$P(\beta|y, \mathbf{X}) \propto P(y|\beta, \mathbf{X}) \cdot P(\beta)$$

Avantages de la régression bayésienne

La régression bayésienne permet de :

- Incorporer des connaissances a priori dans les estimations des coefficients.
- Obtenir une mesure de l'incertitude pour chaque estimation de coefficient à travers la distribution a posteriori.
- Faire des prédictions et calculer des intervalles de prédiction qui prennent en compte l'incertitude de tous les paramètres du modèle.



Formule de Bayes

Rappel du théorème de Bayes :

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

- A : Il pleut,
- B : Il y a des nuages.

Probabilité conditionnelle pour P(Pluie — Nuages) :

$$P(Pluie|Nuages) = \frac{P(Pluie \cap Nuages)}{P(Nuages)}$$

Probabilité conditionnelle pour P(Nuages — Pluie) :

$$P(Nuages|Pluie) = \frac{P(Pluie \cap Nuages)}{P(Pluie)}$$

Égalité des intersections :

$$P(Pluie \cap Nuages) = P(Pluie|Nuages) \cdot P(Nuages) = P(Nuages|Pluie) \cdot P(Pluie)$$

$$P(Pluie|Nuages) = \frac{P(Nuages|Pluie) \cdot P(Pluie)}{P(Nuages)}$$

Classification Bayésienne

La classification bayésienne repose sur l'utilisation de la probabilité conditionnelle et le théorème de Bayes pour prédire la catégorie d'une nouvelle observation. Elle est particulièrement efficace dans les situations où la dimensionnalité des données est élevée ou lorsque les données sont incomplètes.

La formule fondamentale de la classification bayésienne est le calcul de la probabilité a posteriori pour chaque classe C_k donnée une observation $\mathbf{x}$:

$$P(C_k|\mathbf{x}) = \frac{P(\mathbf{x}|C_k) \cdot P(C_k)}{P(\mathbf{x})}$$

où :

- $P(C_k|\mathbf{x})$ est la probabilité a posteriori de la classe C_k étant donné l'observation $\mathbf{x}$.
- $P(\mathbf{x}|C_k)$ est la vraisemblance de l'observation $\mathbf{x}$ dans la classe C_k .
- $P(C_k)$ est la probabilité a priori de la classe C_k .
- $P(\mathbf{x})$ est la probabilité marginale de l'observation $\mathbf{x}$, souvent calculée comme la somme des vraisemblances de $\mathbf{x}$ sur toutes les classes pondérée par leur probabilité a priori.

Classification avec Naive Bayes

Le classificateur Naive Bayes est un modèle probabiliste basé sur le théorème de Bayes, avec une hypothèse simplificatrice d'indépendance conditionnelle entre les caractéristiques données la classe.

Points clés :

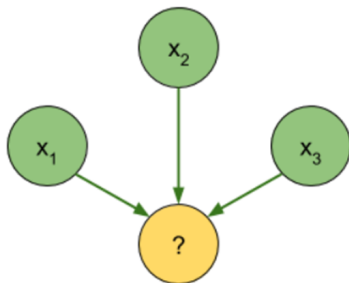
- Facile à construire et particulièrement utile pour de très grands ensembles de données.
- Malgré son hypothèse naïve d'indépendance, il peut être étonnamment efficace.
- Largement appliqué dans la classification de documents, le filtrage anti-spam et les systèmes de recommandation.

Hypothèse d'indépendance dans Naive Bayes

L'hypothèse d'indépendance de Naive Bayes suppose que la présence (ou l'absence) d'une caractéristique particulière est indépendante de la présence (ou l'absence) de toute autre caractéristique, donnée la classe.

Cela simplifie le calcul de $P(x|C_k)$ car :

$$P(x_1, \dots, x_n | C_k) = P(x_1 | C_k) \times \dots \times P(x_n | C_k)$$



Avantages et limites de Naive Bayes

Avantages :

- Efficacité en temps et en espace, même avec de grands ensembles de données.
- Performance robuste et souvent compétitive avec des classificateurs plus sophistiqués.
- Bonne performance sur des données multidimensionnelles et catégorielles.

Limites :

- L'hypothèse d'indépendance peut être irréaliste et peut affecter la performance.
- Moins performant si les caractéristiques sont corrélées.
- Peut être biaisé si l'ensemble de données d'entraînement ne représente pas bien toutes les classes.

Introduction aux réseaux bayésiens

Les réseaux bayésiens sont des modèles graphiques probabilistes qui représentent les relations de dépendance conditionnelle entre un ensemble de variables aléatoires. Ils combinent les principes de la théorie des graphes et de l'inférence bayésienne pour modéliser la structure causale de données complexes.

Caractéristiques clés :

- **Nœuds** : Représentent les variables aléatoires dans le domaine étudié.
- **Arcs** : Indiquent les dépendances conditionnelles entre les variables.
- **Tableaux de probabilité conditionnelle (CPT)** : Quantifient les relations entre les variables.

Les réseaux bayésiens permettent de réaliser des inférences sur des variables inobservées, de prédire les conséquences de interventions et d'apprendre la structure à partir des données.

Inférence dans les réseaux bayésiens

L'inférence dans les réseaux bayésiens consiste à calculer la distribution de probabilité d'un ensemble de variables d'intérêt, étant donné certaines observations. Ce processus est fondamental pour la prise de décision et la prédiction dans les modèles bayésiens.

Méthodes d'inférence :

- **Inférence exacte** : Utilise des algorithmes comme l'élimination de variables ou le message passing (algorithme de Junction Tree) pour calculer précisément les distributions de probabilité.
- **Inférence approximative** : Emploie des méthodes comme le sampling de Monte Carlo et l'inférence variationnelle lorsque l'inférence exacte est impraticable due à la complexité du réseau.

L'objectif est de déterminer $P(X|E = e)$, la probabilité de la variable X étant donné l'évidence $E = e$.

Apprentissage des réseaux bayésiens

L'apprentissage dans les réseaux bayésiens peut être divisé en deux grandes catégories : l'apprentissage des paramètres et l'apprentissage de la structure.

Apprentissage des paramètres : Se concentre sur l'estimation des tableaux de probabilité conditionnelle (CPT) pour un réseau dont la structure est déjà connue. Utilise souvent des méthodes d'estimation maximum de vraisemblance ou bayésienne.

Apprentissage de la structure : Vise à découvrir la structure du réseau (c.-à-d., quels arcs doivent exister) à partir des données. Cela peut être réalisé à l'aide d'approches basées sur les scores, comme le critère d'information bayésien (BIC), ou des approches basées sur la contrainte, qui testent l'indépendance conditionnelle entre les variables.

Apprentissage des paramètres - exemple

L'apprentissage des paramètres consiste à ajuster les valeurs des paramètres d'un modèle pour qu'il s'adapte au mieux aux données. Prenons l'exemple d'un réseau bayésien simple qui prédit la probabilité de jouer au tennis en fonction des conditions météorologiques.

Exemple :

- La structure du modèle est déjà définie : les nœuds représentent des variables comme "Temps" (ensoleillé, nuageux) et "Jouer au tennis" (oui, non).
- L'apprentissage des paramètres ajusterait les probabilités conditionnelles, telles que $P(\text{Jouer au tennis} | \text{Temps} = \text{ensoleillé})$, en se basant sur des données historiques.
- Objectif : Maximiser la correspondance entre les prédictions du modèle et les résultats réels observés dans les données d'entraînement.

Cet apprentissage affinera la capacité du modèle à prédire avec précision si oui ou non une personne jouera au tennis, en fonction des conditions météorologiques observées.

Apprentissage de la structure - exemple

L'apprentissage de la structure cherche à déterminer la meilleure manière d'organiser les relations entre les variables dans un modèle. Considérons l'exemple d'une étude visant à comprendre les facteurs influençant la santé cardiaque.

Exemple :

- Sans connaissances préalables spécifiques, nous disposons d'un ensemble de variables : "Activité physique", "Régime alimentaire", "Poids", "Tension artérielle" et "Santé cardiaque".
- L'apprentissage de la structure peut utiliser des données d'observation pour établir des liens potentiels, par exemple, découvrir si "Poids" agit comme un médiateur entre "Régime alimentaire" et "Santé cardiaque".
- Méthodes : Analyse de corrélation, critères d'information comme BIC, ou algorithmes de recherche structurelle.

Cette approche peut révéler des insights précieux sur les relations causales et les dépendances entre les facteurs de risque et la santé cardiaque, guidant ainsi des interventions préventives plus efficaces.

Chaînes de Markov

Introduction aux chaînes de Markov

Les chaînes de Markov, nommées d'après le mathématicien Andreï Markov, sont un modèle mathématique pour décrire une séquence d'événements où la probabilité de chaque événement dépend uniquement de l'état atteint dans l'événement précédent.

Caractéristiques clés :

- **Propriété de Markov** : La mémoire sans état, où la probabilité de transition vers l'état suivant ne dépend que de l'état actuel.
- **États et transitions** : Définis par un ensemble d'états possibles et les probabilités de passer d'un état à un autre.
- **Matrice de transition** : Chaque cellule P_{ij} de la matrice représente la probabilité de passer de l'état i à l'état j .

Exemple de chaîne de Markov

Considérons une chaîne de Markov simple pour modéliser les prévisions météorologiques avec deux états possibles : Ensoleillé (E) et Pluvieux (P).

États :

- $S = \{E, P\}$

Matrice de transition : La matrice de transition P représente les probabilités de passer d'un type de temps à l'autre d'un jour à l'autre.

$$P = \begin{pmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \end{pmatrix}$$

où P_{ij} représente la probabilité de passer de l'état i à l'état j . Par exemple, $P_{EP} = 0.1$ signifie qu'il y a 10% de chance qu'un jour ensoleillé soit suivi d'un jour pluvieux.

Dynamique :

- Si le temps est ensoleillé aujourd'hui, il y a 90% de chance qu'il soit également ensoleillé demain.
- Si le temps est pluvieux aujourd'hui, les chances sont égales pour que le temps soit ensoleillé ou pluvieux le lendemain.

Introduction aux chaînes de Markov à états cachés

Les chaînes de Markov à états cachés (HMM) sont des modèles statistiques où l'on suppose que le système à modéliser est un processus de Markov avec des états non observés (cachés).

Caractéristiques clés des HMM :

- **États cachés** : Le modèle a des états que l'on ne peut pas observer directement.
- **Observations** : Chaque état caché produit une observation, qui peut être vue ou mesurée.
- **Transitions** : Les états cachés suivent une chaîne de Markov, avec des probabilités de transition spécifiques.

Les HMM sont largement utilisés pour les séquences temporelles où les événements sont liés, tels que la reconnaissance de la parole, la bioinformatique et la finance.

Composants d'un HMM

Un modèle de Markov caché est défini par trois séries de paramètres :

- **Matrice de transition** A : Probabilités de passer d'un état caché à un autre.
- **Matrice d'observation** B : Probabilités de l'observation donnée l'état caché.
- **Distribution initiale** π : Probabilités des états cachés initiaux.

Notation :

- $A = [a_{ij}]$ où a_{ij} est la probabilité de transition de l'état i à l'état j .
- $B = [b_{ij}]$ où b_{ij} est la probabilité d'observer l'observation j dans l'état i .
- $\pi = [\pi_i]$ où π_i est la probabilité initiale de l'état i .

Ces matrices forment le cœur d'un HMM et sont utilisées pour le décodage, l'apprentissage et l'évaluation de séquences.

Algorithmes pour HMM

Les algorithmes clés utilisés dans les HMM sont :

- **Algorithme de Viterbi** : Trouve la séquence la plus probable d'états cachés donnée une séquence d'observations.
- **Algorithme Forward** : Calcule la probabilité de la séquence d'observations donnée jusqu'à un certain point dans le temps.
- **Algorithme Backward** : Calcule la probabilité de la séquence d'observations à venir, en partant d'un état caché.

Ces algorithmes permettent de résoudre les problèmes fondamentaux des HMM : l'évaluation, le décodage et l'apprentissage.

HMM pour la reconnaissance de la parole - Partie 1

Les Modèles de Markov Cachés (HMM) sont au cœur des systèmes de reconnaissance de la parole. Ils permettent de modéliser les séquences sonores temporelles en prenant en compte les variations temporelles et les incertitudes.

Fonctionnement d'un HMM en reconnaissance de la parole :

- Les états cachés représentent les phonèmes ou les unités de son plus petites qui composent la parole.
- Les observations sont des vecteurs de caractéristiques acoustiques extraites des trames sonores.
- Les transitions d'états modélisent la séquence et la durée des phonèmes dans le langage naturel.

Les HMMs utilisent ces composantes pour calculer la séquence de phonèmes la plus probable qui correspond à une séquence d'observations acoustiques, permettant ainsi de transcrire la parole en texte.

HMM pour la reconnaissance de la parole - Partie 2

Applications spécifiques des HMM en reconnaissance de la parole :

- **Modélisation des mots** : Chaque mot peut être représenté par un HMM, avec des transitions entre les états représentant les phonèmes.
- **Coarticulation** : Gérer l'influence d'un phonème sur un autre, qui est crucial pour la reconnaissance précise.
- **Adaptation au locuteur** : Personnaliser les modèles à la voix et à l'accent d'un utilisateur spécifique.

Les HMM sont particulièrement puissants pour la reconnaissance de la parole en raison de leur capacité à capturer les dynamiques temporelles et à gérer la variabilité inhérente à la parole humaine. Ils sont utilisés dans les assistants vocaux, les systèmes de dictée automatique, et les interfaces homme-machine vocales.

Applications des HMM en Marketing

Les Modèles de Markov Cachés sont utilisés en marketing pour analyser les parcours client et prédire les comportements d'achat. Ils permettent de comprendre les séquences de décisions cachées derrière les interactions observables des clients.

Utilisations des HMM en marketing :

- **Segmentation du comportement client** : Identification de séquences de comportement similaires, permettant une segmentation précise des clients.
- **Prédiction de la conversion** : Estimation de la probabilité qu'un client passe à l'achat, basée sur son parcours d'interaction avec divers points de contact marketing.
- **Optimisation de la chaîne de marketing** : Allocation efficace des ressources en analysant les transitions d'états qui maximisent les conversions ou la fidélité.

Deep learning

Brève histoire du deep learning

1940-1980 : Fondations

- 1943 : Neurone artificiel (McCulloch & Pitts).
- 1986 : Rétropropagation (Rumelhart, Hinton, Williams).

1990-2000 : Hiver

- Manque de données et puissance de calcul.
- Progrès en théorie : SVM, modèles bayésiens.

2010-2020 : Renaissance

- 2012 : AlexNet révolutionne la vision par ordinateur.

Depuis 2020 : Multimodalité et LLM

- 2017 : Transformer, base des LLM.
- 2018 - 2020 : GPT-3, BERT.
- 2022 : ChatGPT...

Introduction aux réseaux de neurones

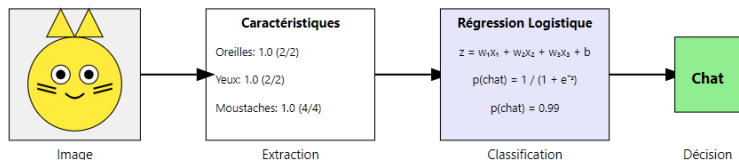
Définition : Les réseaux de neurones sont des modèles computationnels inspirés par le fonctionnement des neurones dans le cerveau humain. Ils sont capables d'apprendre des tâches complexes en modélisant des relations non linéaires entre les entrées et les sorties.

Caractéristiques :

- **Extraction automatique des features** : Capacité d'adaptation et d'extraction des features à partir des données sans programmation explicite.
- **Modélisation non linéaire** : Aptitude à capturer des relations complexes dans les données.
- **Modélisation en grande dimension** : Les modèles de deep learning sont particulièrement adaptés pour les données en grande dimension (images, texte, etc.).
- **Flexibilité** : Applicables à un large éventail de tâches et de typologies de données (images, langage naturel, données graphiques, etc.).

Extraction de features

Extraction de features en machine learning "classique"



Extraction de features en deep learning

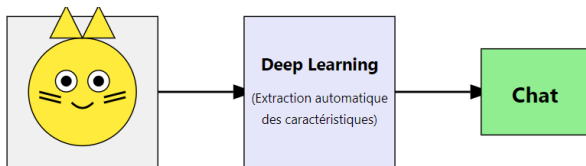
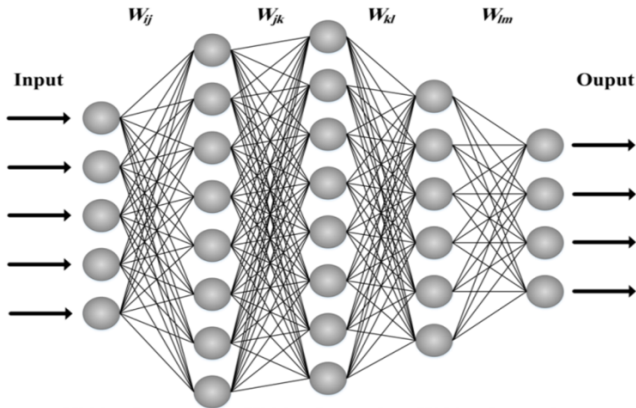


Illustration d'un réseau de neurones classique



Le neurone artificiel

Modèle Mathématique : Chaque neurone artificiel effectue une somme pondérée de ses entrées, ajoute un biais, et passe le résultat à travers une fonction d'activation pour obtenir la sortie.

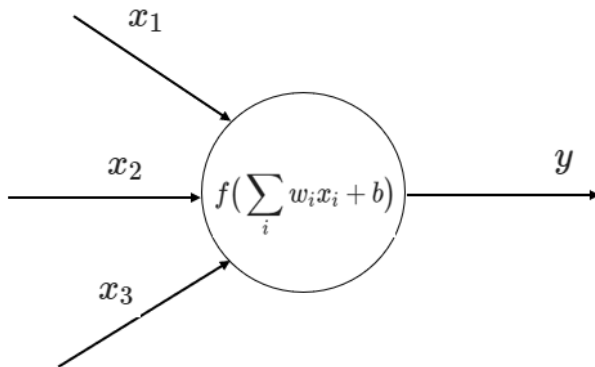
$$z_i = \sum_{j=1}^m w_{ij}x_j + b_i$$

$$a_i = f(z_i)$$

où :

- x_j représente l'entrée du neurone,
- w_{ij} est le poids associé à l'entrée x_j ,
- b_i est le biais du neurone,
- f est la fonction d'activation,
- z_i est le potentiel d'action pré-synaptique,
- a_i est la sortie activée du neurone.

Illustration d'un neurone artificiel



Fonctions d'activation

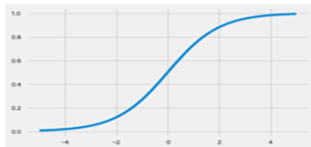
Les fonctions d'activation permettent aux modèles d'apprendre des relations plus complexes en capturant les non-linéarités dans les données.

- **Sigmoïde** : $\sigma(z) = \frac{1}{1+e^{-z}}$, plage de sortie (0, 1), utilisée pour la probabilité dans la classification binaire.
- **Tangente Hyperbolique (tanh)** : $\tanh(z)$, plage de sortie (-1, 1), version centrée et normalisée de la sigmoïde.
- **ReLU (Unité Linéaire Rectifiée)** : $f(z) = \max(0, z)$, non saturante, favorise la convergence rapide et permet d'éviter le problème de disparition des gradients.
- **Leaky ReLU** : $f(z) = \max(\alpha z, z)$, variante de ReLU qui permet un petit gradient lorsque $z < 0$.
- **Softmax** : Utilisée pour la couche de sortie des problèmes de classification multi-classes.

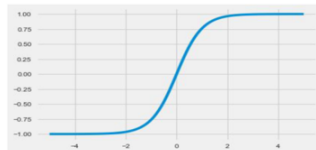
$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Illustration des fonctions d'activation

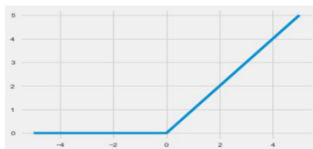
Sigmoïde



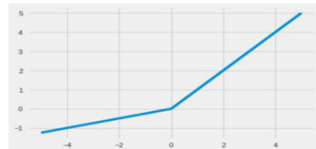
Tanh



ReLU



ReLU paramétrique



Entraînement d'un réseau de neurones artificiel

Principe d'Entraînement : L'entraînement d'un réseau de neurones consiste à ajuster ses poids pour minimiser une fonction de coût qui mesure l'erreur entre les prédictions et les vraies valeurs.

Descente de gradient stochastique (SGD) :

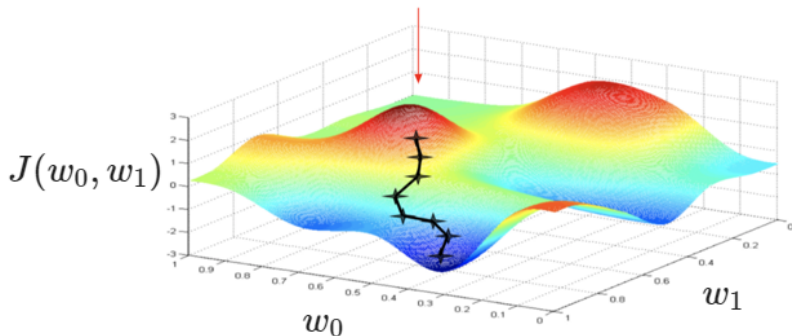
- Méthode d'optimisation utilisée pour mettre à jour les poids du réseau de manière itérative.
- À chaque itération, un sous-ensemble (batch) de données est utilisé pour calculer le gradient de la fonction de coût.
- Les poids sont mis à jour dans la direction opposée du gradient pour réduire l'erreur.

$$w_{new} = w_{old} - \eta \cdot \nabla_w J(w)$$

où :

- w_{old} et w_{new} sont les valeurs des poids avant et après la mise à jour,
- η est le taux d'apprentissage,
- $\nabla_w J(w)$ est le gradient de la fonction de coût par rapport aux poids.

Illustration graphique de la SGD



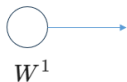
Entraînement des réseaux de neurones profonds

Dans les réseaux de neurones profonds, l'erreur calculée à la sortie du réseau dépend indirectement des poids des couches cachées. Ce lien indirect rend difficile de savoir comment ajuster ces poids pour réduire l'erreur.

Implications pour l'Entraînement :

- **Propagation de l'erreur** : Sans un mécanisme pour propager l'erreur de la sortie vers les couches antérieures, il est impossible de déterminer l'impact de chaque poids sur l'erreur finale.
- **Complexité de l'ajustement des Poids** : Chaque poids dans les couches cachées affecte l'erreur de sortie de manière complexe, nécessitant une méthode précise pour leur ajustement.

Couche 1



...

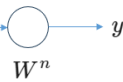
Couche n-2



Couche n-1



Couche n



Rétropropagation du gradient

Il est facile de calculer les gradients correspondant à la dernière couche $\frac{\partial J}{\partial W^{(n)}}$ car l'erreur J dépend immédiatement de $W^{(n)}$.



Examinons maintenant le cas de la couche $n - 1$:

$$\frac{\partial J}{\partial W^{(n-1)}} = \frac{\partial J}{\partial O^{(n)}} \frac{\partial O^{(n)}}{\partial O_{n-1}} \frac{\partial O_{n-1}}{\partial W^{(n-1)}}$$

Surapprentissage dans les réseaux de neurones

Le surapprentissage (overfitting) se produit lorsqu'un réseau de neurones apprend trop bien les détails et le bruit des données d'entraînement, au détriment de sa capacité à généraliser sur de nouvelles données.

Le surapprentissage dans les réseaux de neurones peut se produire pour différentes raisons :

- Complexité excessive du modèle
- Manque de diversité dans les données d'entraînement
- Entraînement prolongé

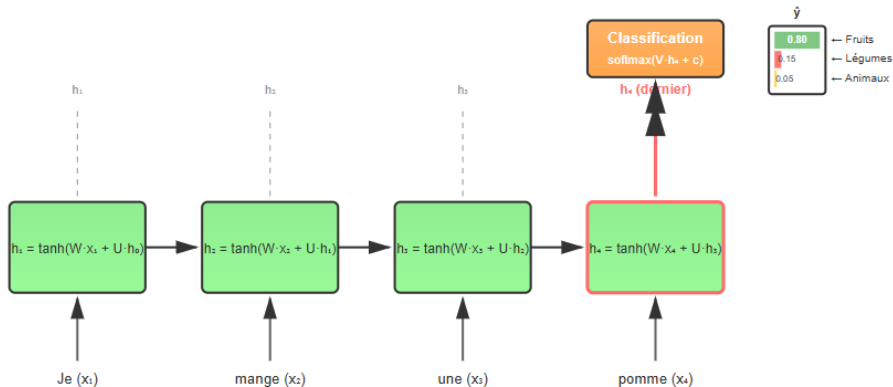
Stratégies pour atténuer le surapprentissage :

- **Régularisation** : Techniques de régularisation telles que L1/L2, Dropout, pour limiter la complexité du modèle.
- **Dropout** : "Eteindre" un ensemble de neurones choisis aléatoirement pendant l'entraînement.
- **Early Stopping** : Arrêt de l'entraînement lorsque la performance sur un ensemble de validation cesse de s'améliorer.
- **Augmentation de Données** : Augmenter la variété des données d'entraînement pour améliorer la robustesse du modèle.

Large Language Models (LLMs)

Anciens modèles pour le traitement du langage : RNN

- **Mémoire à court terme** : Les réseaux de neurones récurrents (RNN) ont la capacité de se "souvenir" d'informations passées grâce à leurs connexions récurrentes.
- **Traitement de séquences** : Ils sont particulièrement adaptés pour des tâches où les données sont séquentielles et où le contexte est important.



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

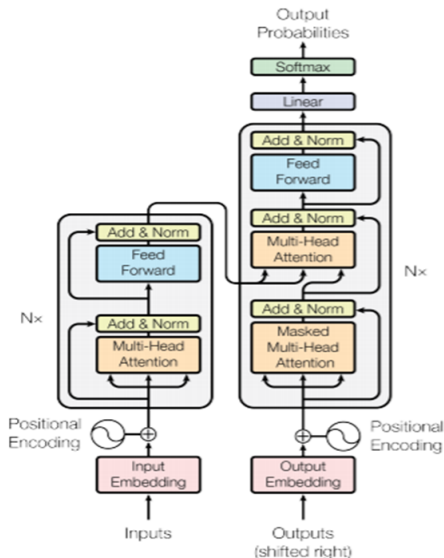
Lukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

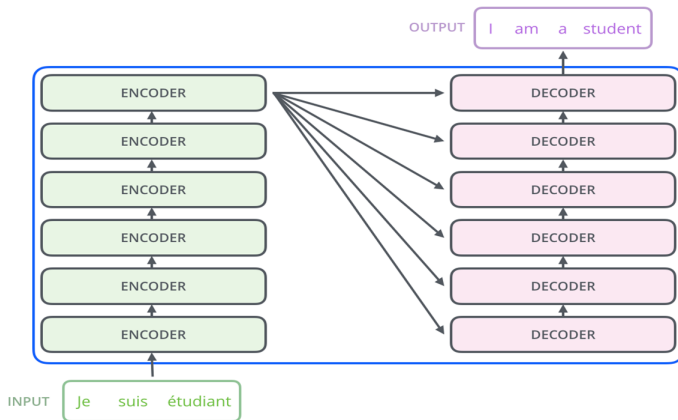
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to

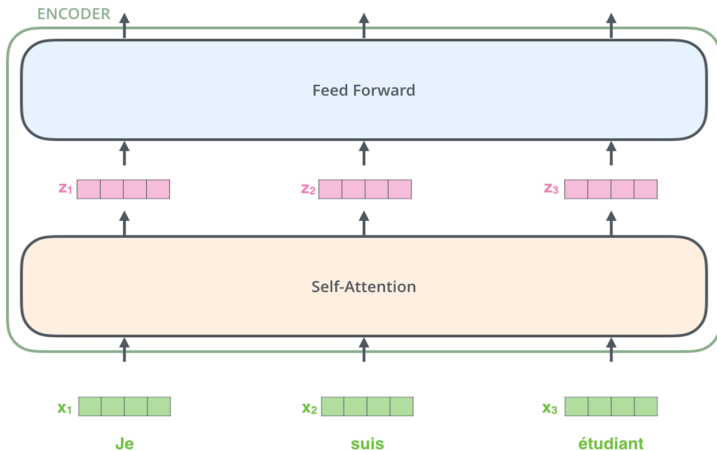
Transformer originel



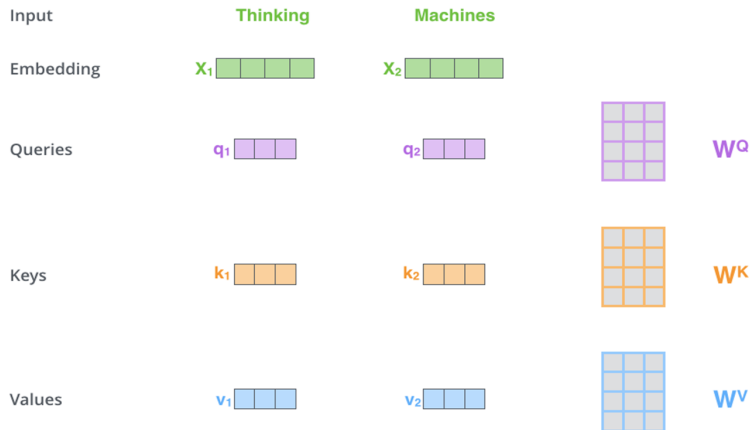
Mécanisme de cross-attention



Mécanisme de self-attention



Fonctionnement du mécanisme de self-attention



Calcul des scores de self-attention

Input

Embedding

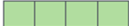
Queries

Keys

Values

Score

Thinking

x_1 

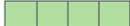
q_1 

k_1 

v_1 

$$q_1 \cdot k_1 = 112$$

Machines

x_2 

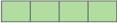
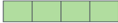
q_2 

k_2 

v_2 

$$q_1 \cdot k_2 = 96$$

Calcul des scores de self-attention

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_2 \cdot k_2 = 96$
Divide by $8 (\sqrt{d_k})$	14	12
Softmax	0.88	0.12

Calcul de la nouvelle représentation

Input

Embedding

Queries

Keys

Values

Score

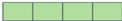
Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax
X
Value

Sum

Thinking

x_1 

q_1 

k_1 

v_1 

$q_1 \cdot k_1 = 112$

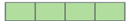
14

0.88

v_1 

z_1 

Machines

x_2 

q_2 

k_2 

v_2 

$q_1 \cdot k_2 = 96$

12

0.12

v_2 

z_2 

Performances du tranformer originel

Entraîné sur WMT 2014 English- German dataset, comprenant près de 4.5 millions de phrases sentence, le WMT 2014 English-French dataset, comprenant près de 36 millions de phrases.

- 8 NVIDIA P10 GPUs
- **Base** : 12 heures
- **Large** : 3.5 jours

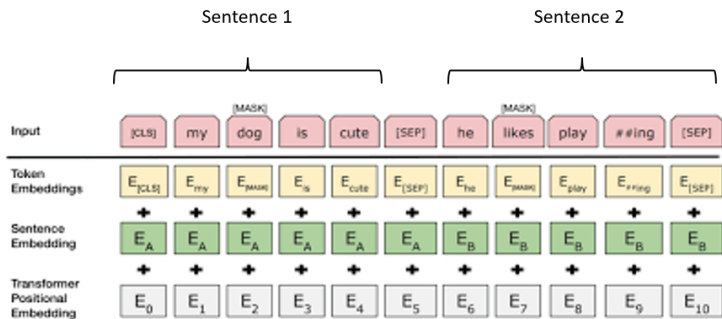
Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

BERT

BERT (Bidirectional Encoder Representations from Transformers) représente une avancée majeure dans le NLP, introduisant une approche novatrice pour la modélisation de langage. Il utilise la bidirectionnalité pour comprendre le contexte des mots, permettant une compréhension plus fine du langage.

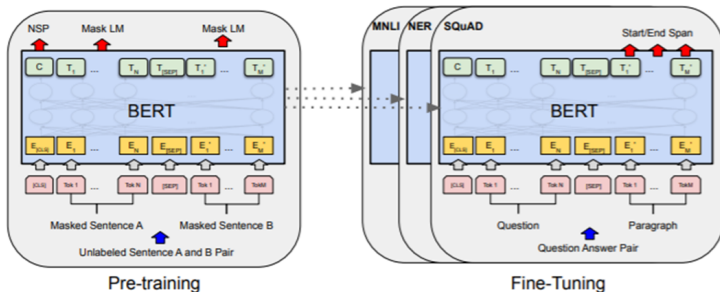
Pré-entraînement



Fine-tuning de BERT pour des tâches spécifiques

Après pré-entraînement, BERT est affiné pour des tâches spécifiques:

- Ajout d'une couche de sortie spécifique à la tâche (classification, NER, QA).
- Fine-tuning de tous les paramètres du modèle pré-entraîné sur le corpus de la tâche.



Performances de BERT

BERT a été pré-entraîné sur le BookCorpus (800 millions de mots) et English Wikipedia (2,500 millions de mots).

Deux modèles :

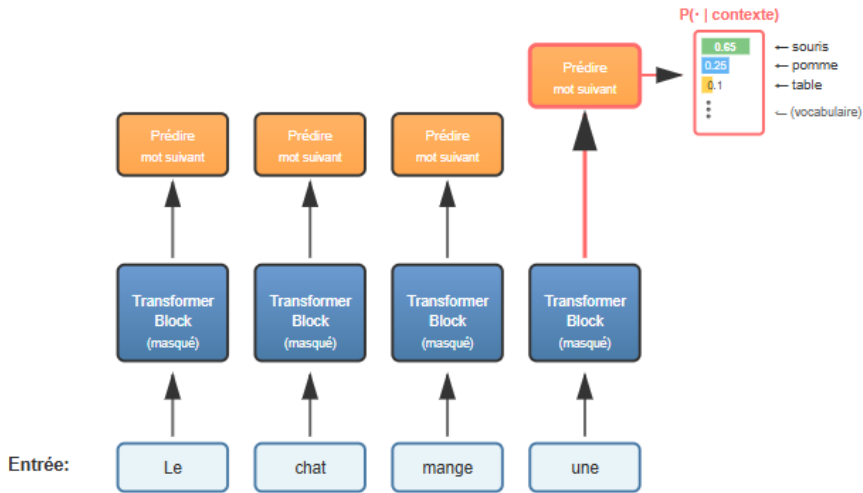
- **BERT Base** : 12 couches avec 110 millions de paramètres.
- **BERT Large** : 24 couches avec 340 millions de paramètres.

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

*Ce que je ne peux pas créer,
je ne le comprends pas.*

– Richard Feynman

Fonctionnement des modèles génératifs



Le paramètre de température

Définition La température (T) contrôle le degré d'aléa dans les réponses générées. Elle agit sur la distribution de probabilité des mots avant l'échantillonnage.

Formule mathématique :

$$P(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

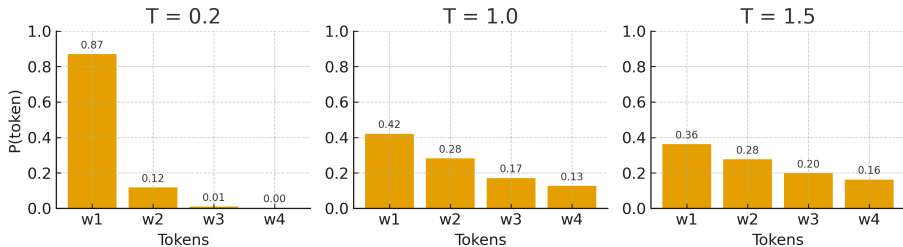
- $T < 1 \rightarrow$ distribution plus concentrée, réponses plus prévisibles
- $T > 1 \rightarrow$ distribution plus plate, réponses plus variées et créatives

Autrement dit :

- Basse température $\rightarrow$ précision et cohérence.
- Haute température $\rightarrow$ imagination et diversité.

Illustration graphique de l'effet de la température

Effet de la température sur la distribution des probabilités



La tokenisation

Problème de départ : Les modèles de langage ne peuvent pas traiter directement du texte brut : les mots, les accents, la ponctuation et les variations sont trop nombreux (plusieurs millions de combinaisons possibles).

La tokenisation : Elle consiste à découper le texte en **unités réutilisables et limitées en nombre**, appelées *tokens*. Chaque token est ensuite représenté par un identifiant numérique que le modèle peut apprendre à prédire.

Intérêt de la tokenisation :

- Réduire la complexité du vocabulaire : on passe de millions de mots à quelques dizaines de milliers de tokens.
- Gérer les mots inconnus ou rares en les décomposant en sous-unités connues.
- Offrir une représentation numérique uniforme du langage, compatible avec les réseaux de neurones.

La composition des données d'entraînement des modèles

Principe général Les modèles de langage sont entraînés sur d'immenses corpus de texte — souvent plusieurs milliers de milliards de tokens — issus de sources très diverses pour couvrir la langue, les faits et les styles.

Sources typiques de données :

- **Web public** : pages web, articles, forums, Wikipédia (ex. Common Crawl).
- **Livres et documents** : œuvres littéraires, articles scientifiques, rapports techniques.
- **Données de code** : dépôts GitHub, notebooks, documentation technique.
- **Conversations et dialogues** : jeux de données de chat, discussions de forums, données annotées par des humains.
- **Données synthétiques** : textes générés par d'autres modèles pour enrichir certaines thématiques.

Équilibre recherché :

- Diversité linguistique et thématique (langues, registres, domaines).
- Qualité et cohérence (filtrage du spam, des doublons, des contenus toxiques).
- Représentation équilibrée entre langage courant, technique et conversationnel.

GPT-3

- Développé par OpenAI, publié en 2020.
- Modèle auto-régressif basé sur l'architecture Transformer.

Caractéristique	Valeur / Détail
Date	2020
Paramètres	175 milliards
Corpus d'entraînement	Environ 300 milliards de tokens
Contexte maximum	2048 tokens
Ressources GPU	Plusieurs centaines de GPU (type V100)
Coût d'entraînement (estim.)	4,6 M\$ à 12 M\$ (selon les sources)

- **Applications** : génération de texte, Q&R, résumé, traduction, assistance à la programmation.

Le few-shot learning

- Les grands modèles (comme GPT-3) peuvent réaliser des tâches **sans être explicitement réentraînés** dessus.
- Le principe repose sur **l'utilisation de quelques exemples** (exemples étiquetés) directement dans le prompt ou l'entrée.
- **Zero-shot** : aucune démonstration fournie, le modèle doit comprendre la tâche à partir de sa connaissance générale.
- **One-shot / Few-shot** : on inclut un nombre très réduit d'exemples (un ou quelques-uns) pour guider le modèle dans la résolution de la tâche.
- Exemple :

English: "cat" → French: "chat"

English: "house" → French: "maison"

English: "car" → French: "voiture"

English: "tree" → French:

ChatGPT

ChatGPT, développé par OpenAI, est un modèle de langage basé sur l'architecture GPT (Generative Pre-trained Transformer) optimisé pour comprendre et générer des dialogues naturels. L'entraînement de ChatGPT se décline selon les étapes

suivantes :

- ➊ **Pré-entraînement sur un corpus volumineux** : Comme GPT-3, ChatGPT est d'abord pré-entraîné sur un vaste ensemble de données textuelles, englobant un large éventail de la littérature disponible sur Internet, pour apprendre une compréhension générale du langage.
- ➋ **Fine-tuning supervisé** : Ensuite, ChatGPT est affiné sur des dialogues spécifiques pour améliorer ses compétences conversationnelles. Cette étape utilise des paires question-réponse et des conversations pour enseigner au modèle des structures de dialogue et des réponses contextuellement appropriées.
- ➌ **Reinforcement Learning from Human Feedback (RLHF)** : Utilisation de techniques de renforcement pour ajuster les réponses du modèle basées sur les préférences et les corrections fournies par des évaluateurs humains, raffinant davantage la pertinence et la naturalité des réponses.

LLMs propriétaires vs Open Source

● Modèles propriétaires

- ChatGPT, Claude, Gemini, Grok...
- Poids non disponibles au public.
- Performances élevées, accès limité par API payante
- Protection intellectuelle stricte (code non accessible)
- Support technique assuré, documentation avancée
- Dépendance aux fournisseurs et coûts potentiellement élevés

● Modèles open source

- Llama, Mistral, Gemma, DeepSeek, Qwen...
- Poids disponibles au public
- Transparence du code, flexibilité d'adaptation
- Performances parfois inférieures, mais amélioration continue
- Gratuit, mais coût de l'infrastructure à gérer
- Exposition aux failles potentielles de sécurité, support communautaire

Conclusion : Choisir entre contrôle, flexibilité et performances optimales.

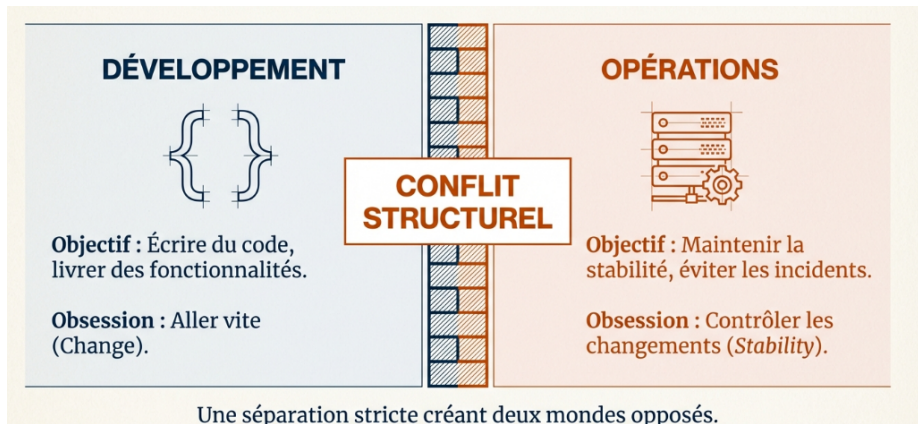
Qu'est-ce que le MLOps ?

Tentative de définition du MLOps

Le MLOps est un ensemble de pratiques et d'outils mis en oeuvre pour automatiser et optimiser le déploiement, la gestion et la maintenance des modèles de machine learning en production.

Pourquoi le DevOps ?

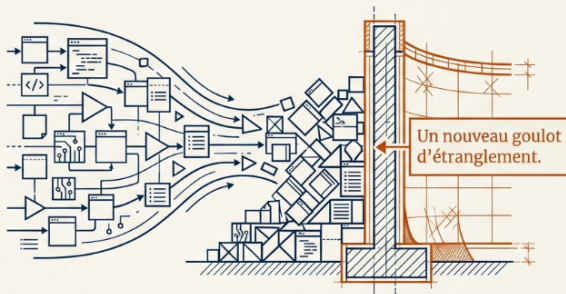
Le DevOps est né d'un conflit structurel entre la phase de développement et celle des opérations.



Arrivée de l'agile : 2005-2008

L'arrivée de Scrum et XP a révolutionné la création de logiciel.

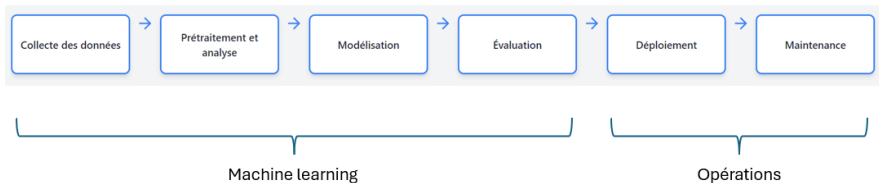
- ✓ Meilleure collaboration entre développeurs.
- ✓ Livraison incrémentale.
- ✓ Qualité du code accrue.



Le logiciel est prêt... mais impossible à déployer rapidement.
L'agilité s'arrête aux portes de la production.

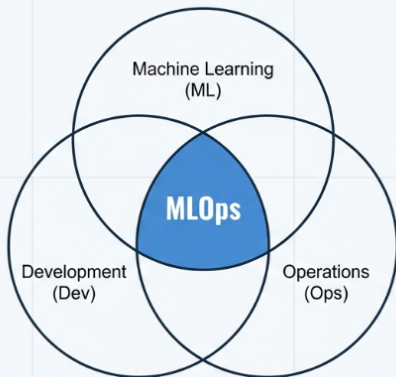
You build it, you run it.

Cycle de vie d'un projet de machine learning



Mais ce processus est en réalité très itératif.

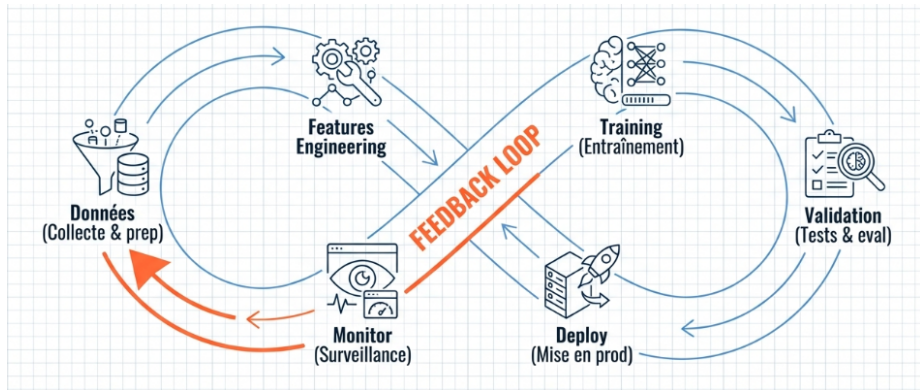
Qu'est-ce que le MLOps finalement ?



**Machine Learning + DevOps
= Production-Ready AI**

Le MLOps est l'ensemble des pratiques pour déployer et maintenir des modèles ML en production de manière fiable, reproductible et scalable.

Cycle de vie avec le MLOps



Différence de nature entre les projets Data et Dev



PROJETS DATA



Nature : Exploratoire, résultats incertains



Objectif : 'Produit fini' flou, amélioration continue



Mentalité : Confort avec l'ambiguïté, exploration



Cycle : Mises à jour fréquentes, attentes parfois irréalistes



Métriques : Statistiques (précision, rappel)



LE FOSSE CULTUREL



PROJETS DEV



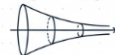
Nature : Déterministe, spécifications claires



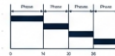
Objectif : 'Produit fini' défini par les specs



Mentalité : Tendance à réduire l'incertitude



Cycle : Traditionnel, estimations plus précises



Métriques : Liées aux fonctionnalités et à la stabilité

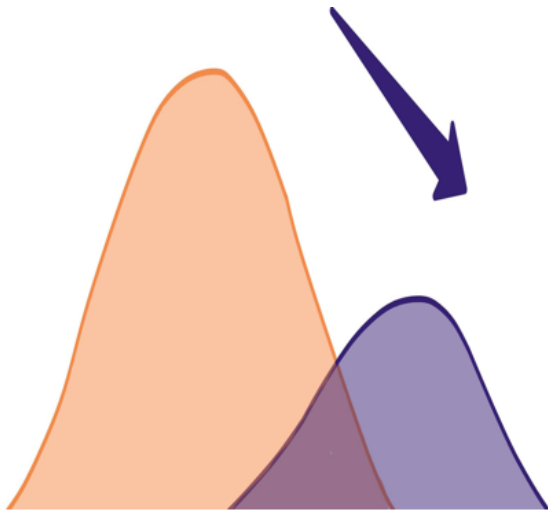


Pourquoi le MLOps ?

Il y a plusieurs problèmes qui émergent lorsque le monde du développement de modèles et celui de la mise en production se rencontrent.



0. Problème de data drift et de concept drift



1. Problèmes de communication

- **Terminologie et langage différents :**

Les data scientists et les développeurs parlent souvent des "mêmes" concepts avec des terminologies différentes. Par exemple, un développeur peut parler d'API ou de scalabilité, tandis qu'un data scientist peut parler de modèles prédictifs et de dérive des données. Cette différence peut créer des incompréhensions.

- **Objectifs mal alignés :**

Les priorités des deux équipes peuvent différer, avec les data scientists axés sur la précision des modèles et les développeurs concentrés sur la stabilité et l'optimisation du code en production. Cette divergence d'objectifs peut entraîner des tensions si les équipes ne communiquent pas clairement leurs attentes respectives.

- **Manque de documentation :**

Les data scientists peuvent fournir des modèles ou des scripts sans documentation complète, ce qui rend difficile pour les développeurs de comprendre le contexte ou les paramètres du modèle lors du déploiement en production.

2. Problèmes techniques

- **Prototypage vs production :**

Le code écrit dans les notebooks n'est pas directement utilisable en production. Il manque souvent de structure, de gestion des erreurs et de tests nécessaires pour être fiable en environnement de production.

- **Scalabilité et performance :**

Les modèles et pipelines développés par les data scientists ne sont pas toujours adaptés pour une exécution à grande échelle en production. Les problèmes de performance (latence, mémoire) peuvent survenir lors du déploiement à grande échelle.

- **Compatibilité des outils :**

Les outils utilisés en exploration (Pandas, scikit-learn) ne sont pas toujours compatibles avec les environnements de production (Kubernetes, Docker, TensorFlow Serving), nécessitant des ajustements ou réécritures du code.

3. Problèmes de gestion des données

- **Qualité et gouvernance des données :**

Les data scientists ont besoin de données propres et de haute qualité, mais si la gouvernance des données n'est pas solide, cela peut entraîner des modèles biaisés ou des erreurs.

- **Accès et gestion des données :**

Il peut y avoir des problèmes de gestion des droits d'accès aux données entre les équipes, ce qui ralentit les projets ou complique les échanges de données entre les équipes data et dev.

- **Versioning des données :**

Les modèles peuvent être entraînés sur une version des données et mis en production avec une autre, ce qui entraîne des incohérences. La gestion des versions des données n'est pas toujours bien synchronisée entre les équipes.

4. Problèmes liés au déploiement et à l'automatisation

- **Déploiement manuel :**

Les modèles sont souvent développés dans des environnements isolés et il peut être difficile de les déployer manuellement en production sans automatisation.

- **MLOps immature :**

Si les pratiques MLOps (intégration et déploiement continus pour les modèles) ne sont pas mises en place, il peut être difficile de maintenir les modèles et de les réentraîner régulièrement en production.

- **Maintenance des modèles :**

Une fois en production, les modèles doivent être surveillés et mis à jour. Sans une automatisation du monitoring et de la reformation, les modèles peuvent devenir obsolètes et imprécis.

5. Problèmes de collaboration organisationnelle

- **Silos organisationnels :**

Les équipes data et dev travaillent souvent dans des silos, ce qui entraîne une mauvaise communication et une collaboration limitée sur les projets communs.

- **Problèmes de priorisation :**

Les équipes data peuvent prioriser des améliorations de modèles, tandis que les équipes dev se concentrent sur la stabilité du système, entraînant des désaccords sur la priorisation des projets.

- **Propriété et responsabilité :**

Il peut être difficile de définir qui est responsable de la performance continue des modèles en production. Les développeurs s'occupent souvent de l'infrastructure, mais les data scientists sont responsables de la précision des modèles.

6. Problèmes liés à la sécurité et à la conformité

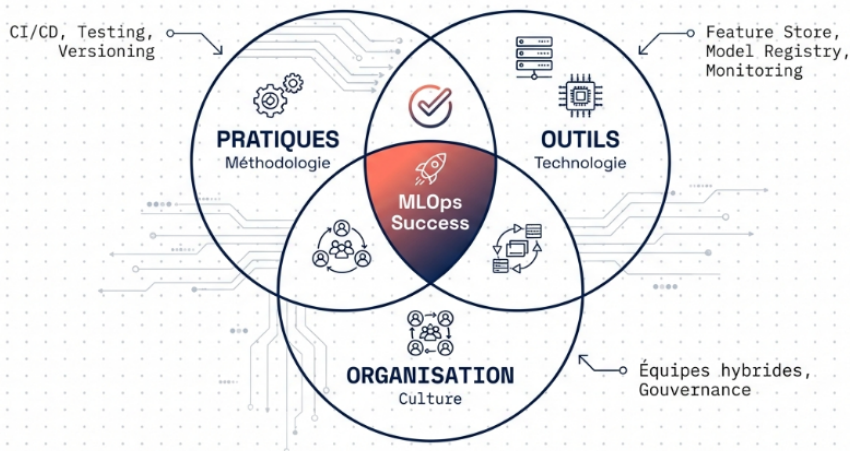
- **Protection des données :**

Les data scientists et les développeurs doivent gérer des données sensibles, ce qui impose des contraintes de sécurité pour éviter les violations de la confidentialité (ex : RGPD).

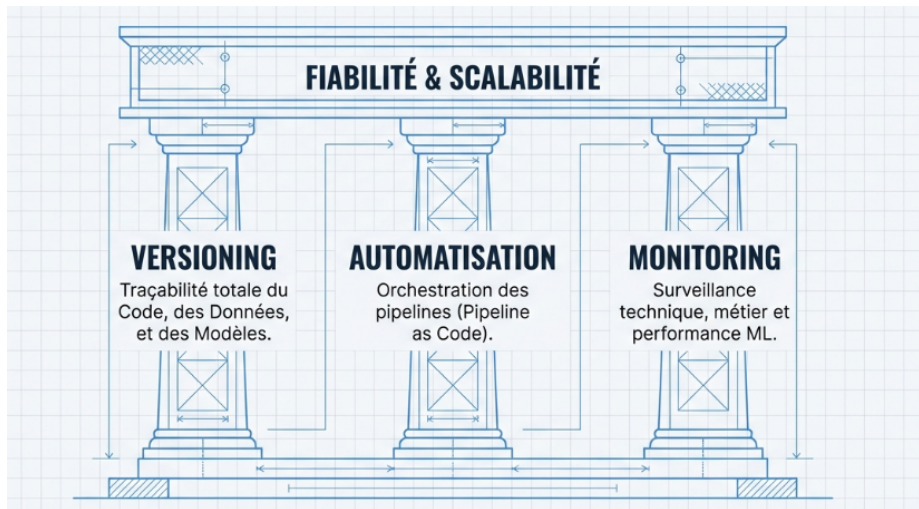
- **Gestion des accès :**

Il est important de s'assurer que seuls les membres autorisés peuvent accéder aux données critiques. Une mauvaise gestion des accès peut entraîner des fuites de données ou des violations de conformité.

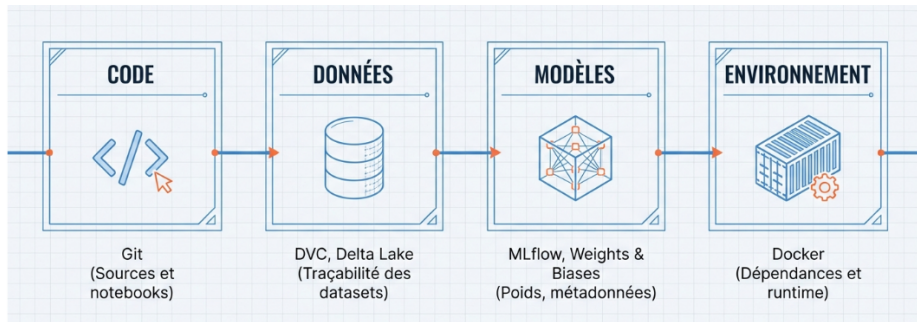
Les trois dimensions du MLOps



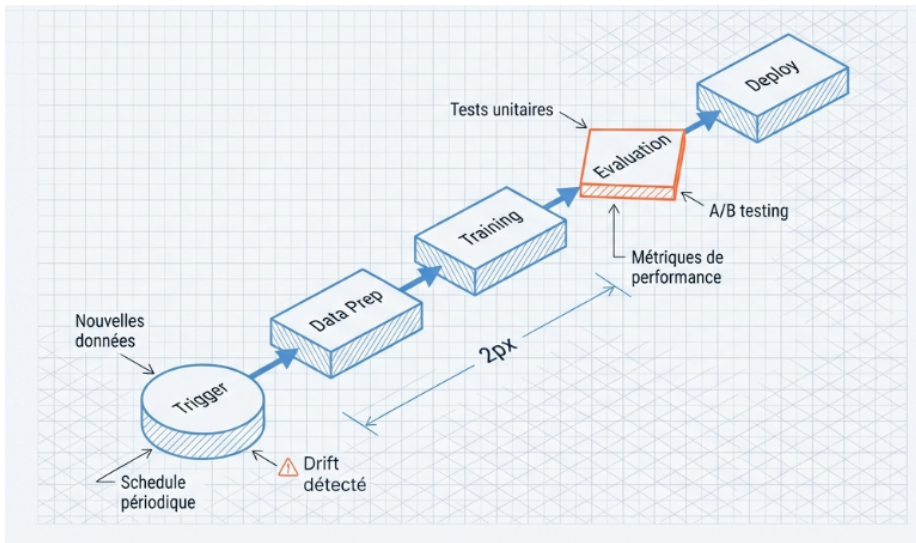
Pratiques : piliers du MLOps



Versioning



Automatisation

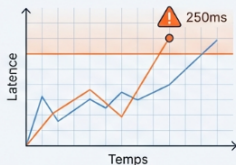


Monitoring

Le monitoring est vital pour la santé du modèle.

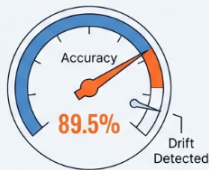
Control Center

MÉTRIQUES TECHNIQUES



- Latence, throughput, erreurs, utilisation ressources (CPU/GPU).

MÉTRIQUES ML

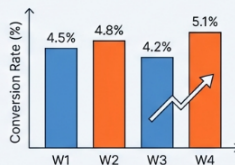


- Accuracy, precision, recall en temps réel.



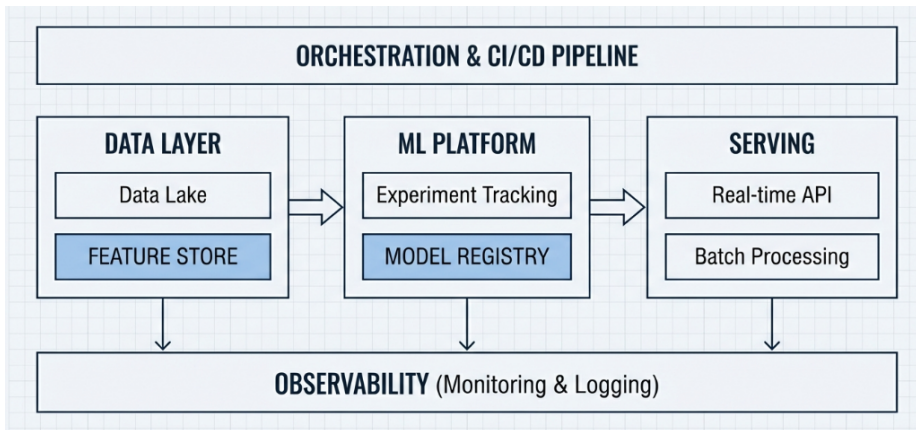
Détection d'anomalies sur la distribution.

MÉTRIQUES BUSINESS



- Taux de conversion, revenu généré, engagement utilisateur.

MLOps de référence



Utils

Versioning



Virtualisation



CI/CD



APIs



Cloud



Plate-formes end-to-end



Les différentes organisations des équipes Data et Dev

Dans le développement et le déploiement des projets de Machine Learning, plusieurs types d'organisation sont possibles pour maximiser la collaboration et l'efficacité entre les équipes de data science, de développement et d'ingénierie.

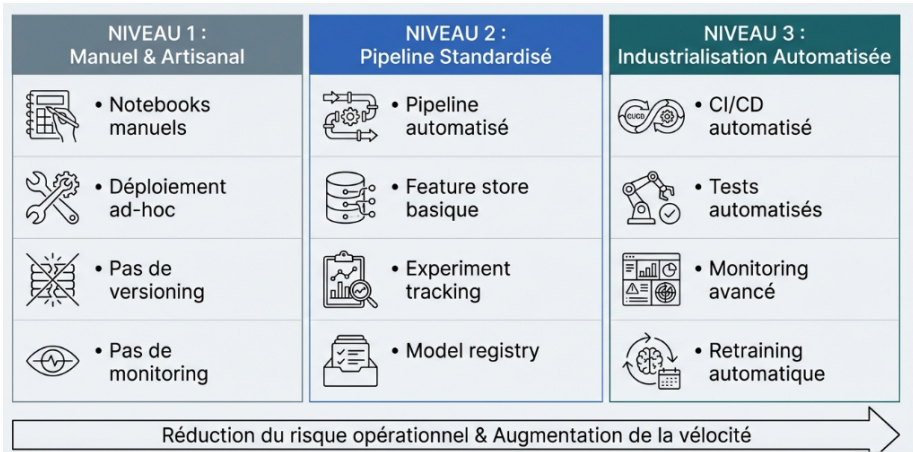
- ➊ **Approche en silos (traditionnelle)** : Les équipes travaillent de manière indépendante, avec transfert de responsabilités des data scientists aux devs.
- ➋ **Approche collaborative (cross-fonctionnelle)** : Les équipes collaborent dès le début du projet et tout au long du cycle de vie du modèle.
- ➌ **Approche DevOps/MLOps** : Automatisation des processus d'entraînement, de déploiement et de surveillance des modèles à travers des pipelines CI/CD.
- ➍ **Équipe centralisée (Center of Excellence)** : Une équipe dédiée centralise les compétences et les processus pour tous les projets de ML de l'entreprise.
- ➎ **Squads dédiés ou équipes produits (Product Teams)** : Des équipes autonomes et multidisciplinaires sont responsables du développement de produits spécifiques intégrant des modèles ML.
- ➏ **Approche par chapitre et guildes (Spotify Model)** : Une combinaison d'équipes autonomes (squads) et de groupes d'experts (chapitres) partageant des pratiques et des standards communs.

Organisation : principaux métiers de l'IA

Les métiers de l'IA sont en constante évolution et ont subi une évolution assez importantes ces dernières années. On peut distinguer cependant les métiers suivants :

- **Data scientist** : il est au cœur du projet IA. Son principal rôle consiste à traduire le besoin métier en une problématique technique pour élaborer un modèle de ML ou autre, qu'il doit tester et valider avec les métiers. Il est donc normalement amené à travailler étroitement avec les métiers.
- **Machine Learning Engineer** : son rôle consiste à adapter le code développé par le data scientist pour le déployer, souvent sur une infrastructure Cloud (AWS, GCP, Azure, etc.). Il doit ensuite veiller au bon fonctionnement de l'application.
- **Data Engineer** : il est responsable de l'infrastructure liée au projet (pipelines de données, etc.).
- **Data analyst** : son rôle est d'analyser les données pour répondre à des questions métier.
- **Data architecte** : il conçoit l'architecture de la solution qui intègre éventuellement de l'IA (bases de données, back-end, front-end, services Cloud, etc.), notamment quand celle-ci est complexe.

Niveaux de maturité du MLOps



La maturité dépend de plusieurs facteurs

Les approches MLOps peuvent être différentes d'une entreprise à une autre et dépendent de plusieurs facteurs :

- Infrastructure de l'entreprise.
- Organisation de l'entreprise.
- Ressources disponibles en termes de compétences.
- Projets mis en oeuvre.
- Maturité de l'entreprise du point de vue du ML.
- Etc. etc.

Bibliographie

- Hastie, T., Tibshirani, R., Friedman, J. H., Friedman, J. H. (2009). The elements of statistical learning: data mining, inference, and prediction (Vol. 2, pp. 1-758). New York: springer.
- Russell, S. J. (2010). Artificial intelligence a modern approach. Pearson Education, Inc..
- Ng, A. (2000). CS229 Lecture notes. CS229 Lecture notes, 1(1), 1-3.
- Azencott, C. A. (2019). Introduction au machine learning. Dunod.
- Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep learning. MIT press.
- Vapnik, V. (1999). The nature of statistical learning theory. Springer science business media.
- Cortes, C., Vapnik, V. (1995). Support-vector networks. Machine learning, 20(3), 273-297.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... Amodei, D. (2020). Language models are few-shot learners. Advances in neural information processing systems, 33, 1877-1901.

Merci de votre attention

redha.moulla@axia-conseil.com